

**An Agentic Retrieval-Augmented Forecasting
Framework for India's Automotive Market:
Integrating Multi-Document Intelligence, Policy
Analysis, and Time-Series Modeling**
A PROJECT REPORT

Submitted by

RAVIKANTH AMAJALA [Reg No: RA2412044015067]

Under the Guidance of

DR. K. VENKATESH

Associate Professor, Department of Data Science and Business Systems

In partial fulfilment of the requirements for the degree of

MASTER OF TECHNOLOGY

in

BIG DATA ANALYTICS



**DEPARTMENT OF DATA SCIENCE AND BUSINESS
SYSTEMS**

COLLEGE OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR – 603 203

MAY 2025

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY**KATTANKULATHUR – 603 203****BONAFIDE CERTIFICATE**

Certified that this M.Tech project report titled “**CLASSIFICATION AND DETECTION OF AGGRESSIVE DRIVING BEHAVIOUR**” is the bonafide work of **Ms. Nivetha Jayakumar [Reg. No.: RA1811027010013]** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion for this or any other candidate.

Dr. A. MURUGAN
SUPERVISORAssociate Professor
Department of Data Science and Business
Systems**Dr. V. KAVITHA**
HEAD OF THE DEPARTMENTProfessor
Department of Data Science
and Business Systems**INTERNAL EXAMINER****EXTERNAL EXAMINER**



DEPARTMENT OF DATA SCIENCE AND BUSINESS SYSTEMS

SRM Institute of Science and Technology Own Work Declaration Form

Degree/ Course : M.Tech in Big Data Analytics
Student Names : Nivetha Jayakumar
Registration Number : RA1811027010013
Title of Work : Classification and Detection of Aggressive Driving Behaviour

I hereby certify that this assessment compiles with the University's Rules and Regulations relating to Academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

I confirm that all the work contained in this assessment is my own except where indicated, and that I have met the following conditions:

- Clearly references / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc.)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that I have received from others (e.g. fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook / University website

I understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

DECLARATION:

I am aware of and understand the University's policy on Academic misconduct and plagiarism and I certify that this assessment is my own work, except where indicated by referring, and that I have followed the good academic practices noted above.

Nivetha Jayakumar RA1811027010013

ACKNOWLEDGEMENT

I express my humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support. I extend my sincere thanks to Dean-CET, SRM Institute of Science and Technology, **Dr. M. Leenus J Martin**, for his invaluable support.

I wish to thank **Dr. Revathi Venkataraman**, Professor & Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work. I sincerely appreciate **Dr. V. Kavitha**, Professor and Head, Professor, Department of Data Science and Business Systems, SRM Institute of Science and Technology, for her guidance and support that has been invaluable to my success. I am incredibly grateful to my Program Coordinator, **Dr. S. Ganesh Kumar**, Professor, Department of Data Science and Business Systems, SRM Institute of Science and Technology, for his suggestions and encouragement at all the stages of the project work.

I want to convey my thanks to my Project Coordinator, **Dr. A. ShobanaDevi**, Associate Professor and the Panel Head, **Dr. A. Murugan**, Associate Professor Department of Data Science and Business Systems, SRM Institute of Science and Technology, for their inputs during the project reviews and support.

I register my immeasurable thanks to my Program Manager, **BEN KINGSLEY**, Program Manager, Great Learning for leading and helping us to complete the course.

My inexpressible respect and thanks to the guide, **Dr. A. Murugan**, Associate Professor, Department of Data Science and Business Systems, SRM Institute of Science and Technology, for providing me with an opportunity to pursue my project under his mentorship. He provided us with the freedom and support to explore the research topics of my interest. His passion for solving problems and making a difference in the world has always been inspiring.

I sincerely thank the Department of Data Science and Business systems staff and students, SRM Institute of Science and Technology, for their help during my project. Finally, I would like to thank parents, family members, and friends for their unconditional love, constant support, and encouragement.

Nivetha Jayakumar [RA1811027010013]

TABLE OF CONTENTS

<i>C.NO.</i>	<i>TITLE</i>	<i>PAGE NO.</i>
	Abstract	ix
	List of Tables	x
	List of Figures	xi
	List of Symbols and Abbreviations	xiii
1.	INTRODUCTION	1
1.1	General	1
1.2	Purpose	2
1.3	Scope	4
1.4	Artificial Intelligence, Machine Learning and Deep Learning	4
1.5	Computer Vision	9
2	LITERATURE REVIEW	10
2.1	DriftNet	10
2.2	Previous Frameworks	12
2.3	ResNets	12
2.4	EfficientNets	13
2.5	RandAugment	15
2.6	Hausdorff Distance	16
3	PROPOSED METHODOLOGY	18
3.1	Dataset	18
3.2	Feature Extractor	22
3.3	Classifier	29
3.4	Detection	40
3.5	Algorithm	42
4	RESULTS	44
5	CONCLUSION	49
6	FUTURE SCOPE	50
7	REFERENCES	51

APPENDIX 1	54
APPENDIX 2	57
PAPER PUBLICATION STATUS	71
PLAGIARISM REPORT	72

ABSTRACT

Aggressive driving is a menacing behaviour that questions the safety and puts human life at risk. Several methodologies are currently used to identify various elements like speeding, reckless driving and traffic violation. Advancements in deep learning and computer vision have made it possible to train highly complex deep neural networks for identifying such anomalous behaviour. However, most methodologies are computationally costly, and the software require high computing servers for being hosted. We aim to propose an efficient method that incorporates various modules such as driving pattern classification, and vehicle detection by incorporating state-of-the-art convolution neural network architectures, such as the ResNet-50 and the EfficientNet-B7, and a simple recurrent neural network model for feature extraction and classification respectively. We have also leveraged computer vision tools to be applied to the detection of vehicles involved in rash driving. Methods such as thresholding, frame differencing, processing of contours using area and Hausdorff distance were utilised. Our approach produced high training and validation accuracies of 90.62% and 90.17% respectively with merely 144,026 trainable parameters. The aim of this experiment is to prove that identifying aggressive driving behaviours can be done in a cost-effective manner.

LIST OF TABLES

2.1.1	Parameters of DriftNet.....	11
2.4.1	Parameters EfficientNet.....	15
3.3.1	Different types of RNNs.....	30
3.3.2	Attributes of the SimpleRNN() function.....	31
4.1	Parameters and their values before and after tuning.....	47

LIST OF FIGURES

1.4.1	Types of Machine Learning	
	(a) Supervised Learning	6
	(b) Unsupervised Learning	6
	(c) Reinforced Learning	7
1.4.2	(a) A single-layer perceptron. (b) A multi-layer perceptron	8
1.5.1	Applications of Computer Vision.....	9
2.1.1	Comparison of three Architectures (a)C3D, (b)ConvLSTM, (c) DriftNet.....	11
2.3.1	Figure showing three different ImageNet-scaled network architectures (a)VGG-19, (b)plain network with 34 parameter layers and (c)ResNet with 34 parameter layers.....	13
2.4.1	Scaling of a baseline model (a) using conventional methods – width (b), depth (c), resolution of the input images (d), and the proposed compound scaling of EfficientNet across all three dimensions (e).....	14
2.5.1	Examples of images augmented by RandAugment with number of transformations (N)=2 and the distortion magnitude(M)=9 (a), 17(b), and 28(c) for ResNet-50, EfficientNet-B5 and EffientNet-B7 respectively.....	16
2.6.1	Image depicting the calculation of Hausdorff distance between two polygons curves.....	17
3.1.1	Sample Frames from the Dataset.....	20
3.1.2	First five records of the data frame containing the paths to the videos in the training set along with their labels	21
3.1.3	First five records of the data frame containing the paths to the videos in the testing set along with their labels.	21
3.2.1	Architectural details of the EfficientNet-B7	
	(a) defining the layers of the stem and the final blocks.....	23
	(b) defining the layers of the modules used in the final EfficientNet-B7 network.....	23

(c) defining the final architectural design of the EffientNet-B7 with the modules and blocks of layers defined earlier.....	24
3.2.2 Architectural details of the ResNet-50.....	25
3.2.3 Differences between Max Pooling and Average Pooling.....	27
3.2.4 Proposed Architecture of the feature extractor CNN.....	28
3.3.1 Architectural details of (a) a traditional RNN and (b) the individual blocks of recurrent connections.....	29
3.3.2 Sigmoid function.....	33
3.3.3 Tanh function	34
3.3.4 Softmax function.....	35
3.3.5 Curves of predictions and true values for different classes of model performance	37
3.3.6 Proposed Architecture of RNN Classifier.....	39
3.4.1 Frames Post Thresholding and Dilation.....	41
3.5.1 Activity Diagram for the combined model.....	43
4.1 Training and validation accuracies and loss curves before parameter tuning.....	45
4.2 Training and validation accuracies and loss curves of the model trained on the UCF-101 dataset before parameter tuning.....	45
4.3 Training and validation accuracies and loss curves after tuning.....	46
4.4 Formulas for precision, recall and f1-score.....	47
4.5 Precision, recall and f1-scores for the two classes (aggressive and non-aggressive)	47
4.6 Annotated Frame Post Detection.....	48

LIST OF SYMBOLS AND ABBREVIATIONS

US	United States of America
CCTV	Closed-circuit television
3D	Three Dimensional
PC	Personal Computer
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
AI	Artificial Intelligence
ML	Machine Learning
DNA	Deoxyribonucleic Acid
C3D	Convolutional 3 Dimensional
LSTM	Long Short-Term Memory
ConvLSTM	Convolutional Long Short-term Memory
JTSM	Joint Time series modelling
FLOPS	Floating-point operations per second
RMSProp	Root Mean Squared Propagation
SiLU	Sigmoid Linear Units
MIT	Massachusetts Institute of Technology
CADP	Car Accident Detection and Prediction
VIRAT	Video Image Retrieval and Analysis Tool
RGB	Red Green Blue
BGR	Blue Green Red
i.e.	id est
e.g.	exempli gratia
Eq	Equation
GPU	Graphics Processing Unit
RAM	Random-Access Memory
OpenCV	Open source Computer Vision library

CHAPTER 1

INTRODUCTION

1.1. General

As of 2019, reports state that there are around 295.8 million active vehicles running on the public roads of India. Since 2012, the average number of newly issued drivers' licenses in the country is around 11.5 million. In 2016, Brookings Analysis reported 76% of daily commuters in the United States of America drive their personal vehicles, which equates to over 115 million automobiles on roads. Since these reports were published, most of these numbers have trended upwards. Resulting in the greatest number of active drivers of all time.

These numbers by itself does not tell the entire story, the story of the ugly side of the “everyday commute”. Traffic management with the target of enhancing road safety has always been immensely expensive. In the US for the fiscal year of 2022-2023, the government has allocated \$2.7 billion for the road safety aspect, which is 6% of the budget allocated to the total expenditure on highways. This may seem like a meagre percentage but is significant when compared to the allotment for road safety being just 0.26% of the budget committed to highway expansion by the Indian government. This is especially concerning considering India accounts for approximately 11 percent of global fatalities due to road crashes.

This is the driving force of our project. To help understand and identify an aspect of driving behaviours in an attempt to provide cost effective countermeasures. Aggressive driving poses a significant risk to the safety of road users. Approximately 1.3 million people die each year because of road traffic crashes and cost countries around 3% of their gross domestic product. Though some of them can be attributed to structural and mechanical failure, most cases are a result of negligent driving.

Gone are the days where there is a need to spend countless man-hours observing and directing the masses on roads across the country. Over the last few decades, rapidly growing countries like India have started shifting towards and depending on digital means of policing traffic. This has ushered in the new age of understanding and identifying driving behaviors of drivers and provides the opportunity to enforce safety laws on a low-level platform. There is a downside to this transformation of processes into the digital age. Costs.

From a report by Mathrubhumi, over 700 closed-circuit cameras have been installed in the state of Kerala, India, for the whopping price of \$30 million. This is of course the highest end equipment and delivers with incredibly high accuracy and consistency, but we believe these results can be achieved with far less. Using Deep Learning techniques and a vast and deep pool of data, our project is proof of a sustainable solution.

1.2. Purpose

Through this project, we aim to deliver a feasible, and viable method to understand and identify the reckless behaviours of individuals on highways and freeways. We aimed to create a proof of concept to substantiate our claim that aggressive driving habits can be distinctly categorized from that of normal and/or acceptable behaviours, while maintaining a cost-effective approach.

There are many existing methods that can accomplish parts of our aim. There are highly advanced cameras and sensors that enable multiple-car tracking to be done on a large scale. A prime example of this is the usage of closed-circuit traffic camera footage that is being collected in every country. However, the implementation of these networks have proven to be massively taxing on the road and safety budgets. Often times governments seek input of the hardware and the software from third-party companies.

We aim to fill the void of inexpensive, yet reliable system options that can be deployed with ease. Our project is one that proves the highest end, state-of-the-art technology is not necessary to achieve the safety goals that we set. With a wide enough data pool, we have been able to

show that a home computer can achieve consistent results to help us understand aggressive driving behaviours and drive safety forward.

Our project is on nearer side of the complexity spectrum when it comes to hardware and essential conditions. It's proven that with just a standard CCTV camera capturing standard footage at a standard frame rate fused with the deep learning techniques we have induced can identify high risk situations and clusters of vehicles on highways and freeways. We have achieved this without the use of high-powered computing systems, nor with the availability of standardized footage from the wide network of traffic surveillance. This is proof of the computational efficiency. The engine of our project promises growth and guarantees development and higher efficiency with more and more video footage. With more data being fed through the training aspect, the results are sure to become more accurate as well as consistent.

The most common deep learning approaches to video processing and classification are using a 3D-convolution neural network architecture and using a combination of a feature extraction algorithm followed by its classification by a time-series model [1] [2]. In this study, we aim to optimize the latter to produce a computationally inexpensive classifier that can be trained and utilised on a home PC. To achieve this, we have compared a handful of efficient algorithms, CNNs and RNN architectures. Since the training cost of an RNN is higher than that of running feature extraction algorithms, we have increased the complexity of the feature extraction module by combining two of the most precise pre-trained CNN architectures, the EfficientNet-B7 and the ResNet-50, while simultaneously decreasing the complexity of the RNN classifier. The compound combination of the CNNs would help extract the most vital and highly detailed features that are key in classifying driving patterns. Furthermore, parameter tuning, and weight initializations are experimented with to determine those that result in a model that attains the maximum performance criteria for our problem statement. For detecting the vehicles included in the event, we have made use of computer vision methods and directed Hausdorff distance to determine those that are dangerously close or have diverging tangents of movement [3]. This further eliminates the painstaking process of annotating videos of the dataset for a bounding box regressor to be trained on.

1.3. Scope

For the project on hand, we have collected CCTV footage of highways with flowing traffic consisting of a wide range of vehicles with different characteristics. Consistently aggressive patterns include Cutting Lanes, passing on shoulders (exit ramps / entry ramps), Speeding and/or Unusual Acceleration, Lateral Displacement (Lane Switching), Sharp Turns, Unusual Deceleration (excessive braking). However, patterns and variables such as Congestion and Traffic and Factors such as Age, Gender, Presence/Absence of passengers, Honking and tailgating will not be included in the scope of our project.

Our solution requires video that is recorded from a static point of view. This is to enable consistent results which wouldn't be possible with large disparities in standards and formats. Videos for our project are required to have vehicles in question to be in frame for at least a few seconds to enable our model to understand the characteristics and trajectories of the vehicle. Videos which contain a vehicle being in frame for longer durations will ensure better accuracy in terms of understanding the behaviors of the particular vehicle caused by the driver.

1.4. Artificial Intelligence, Machine Learning and Deep Learning

Artificial Intelligence is a subset of computational theory and logic which focuses on the development of computer systems and algorithms designed to perform tasks that usually require human intelligence, logical thinking and expertise. These programs and applications imitate the behaviour of the human mind, hence require superabundant knowledge related to all the variables of the problem. These include the various objects, their properties, differentiating categories and the relationships between them. Artificial Intelligence can be broadly classified into two types depending on the kind of problems they are developed to solve;

- **Vertical AI:** These AI algorithms focus on learning to solve a single problem. They are usually restrictively programmed to follow the instructions for a single, automated and repetitive task. For example, scheduling meetings and calls on a daily basis,

automatically updating the database with information obtained from a single external source.

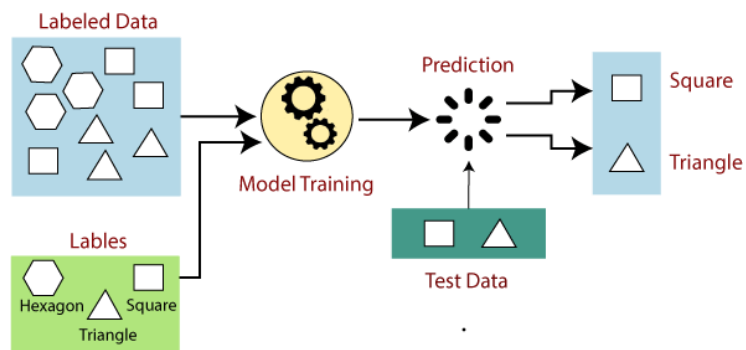
- Horizontal AI: These AI algorithms focus on broader problem statements. Applications classified under this type of AI have the ability to handle multiple tasks and cater to all multiple needs of their user with a single logic and setting. For example, virtual assistants like Siri, Cortana and Alexa that tackle multiple tasks for their users.

Machine Learning (ML), though often confused and interchangeably used with Artificial Intelligence, is a subset of artificial intelligence and is the science of developing machine intelligent algorithms that help the machine learn from existing and past data. The algorithms achieve this by identifying and learning the various relationships between individual features of the data, looking for common patterns, responding to different situations outside of their programming restrictions and makes predictions accordingly.

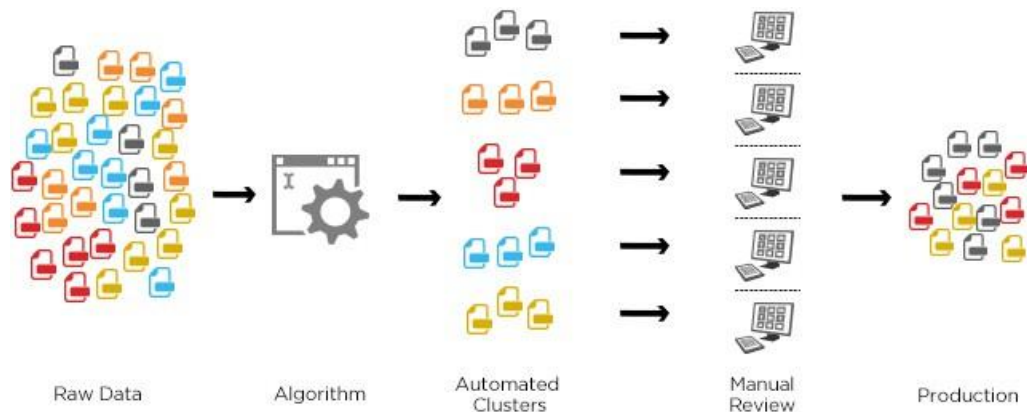
Machine Learning can be broadly classified into three categories (figure 1.4.1) based on the kind of data they use to learn and the type of outputs produced by them. They are:

- Supervised Learning: In this type of learning, the datasets that the machine utilises to learn are entirely structured and labelled. The datasets have a set of input features and their corresponding outputs and are given to the ML algorithm as inputs for training. While training, the model learns by mapping its predictions to the true predictions present in the dataset, evaluates its performance, and automatically updates its learning curve to achieve better scores of the performance metrics. For example, classification and regression tasks such as image classification and commodity price prediction respectively.
- Unsupervised Learning: In this type of learning, the datasets are not mapped with their outputs. The models are trained on a dataset consisting of several features and are expected to produce outputs by learning and identifying common patterns without conforming to a certain “correct answer”. For example, clustering algorithms used for customer segmentation, DNA pattern recognition and grouping in evolutionary biology.

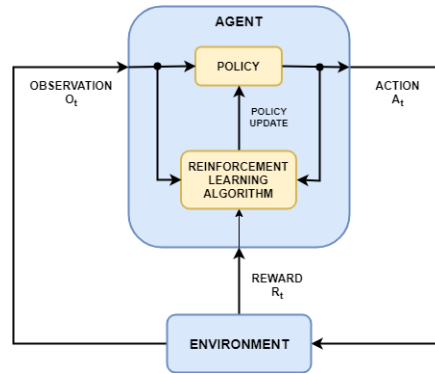
- **Reinforcement Learning:** In this type of learning, the algorithm updates its learning curve by studying the features of the learning problem, environment and the behaviour of the agent in order to maximise its performance. It works on a action-reward based system by learning any method that is well suited to solve the problem, while simultaneously correcting any method that does not. The ultimate goal is reached efficiently and rapidly and the machine makes sure to learn only those methods that facilitate this. For example, autonomous driving and gaming technologies.



(a)



(b)



(c)

Figure 1.4.1.: Types of Machine Learning (a) supervised learning (b) unsupervised learning and (c) reinforcement learning

Deep Learning is a subset of machine learning, where the machine intelligent models are essentially imitations of the human brain. The model, called an Artificial Neural Network, is similar to the human brain where connections exist between the individual units (neurons). However, unlike the human brain, the individual perceptrons are restricted to forming a set number of connections with certain layers and are allowed to propagate data only in restricted directions defined according to the type of the neural network.

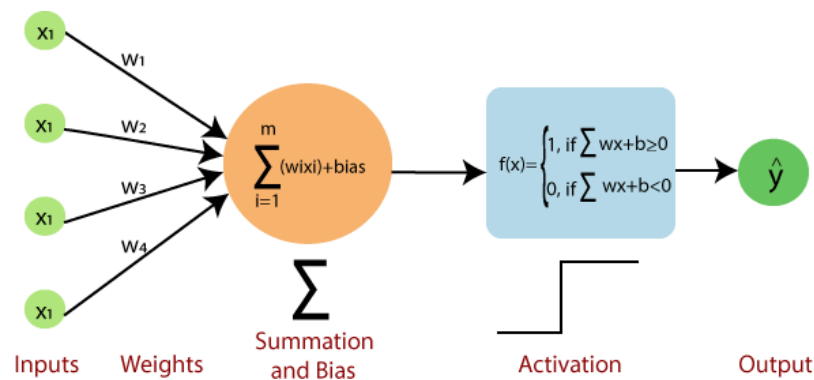
Neural networks can be designed and trained to perform any kind of task that machine learning algorithms can and are more robust than ML models. These networks have the ability to learn from raw data, grasp the various features as the data is passed on through the different layers while simultaneously compressing the data and produce the output from these set of features extracted during propagation.

A perceptron, which is an individual unit of a layer of a neural network, is a complex mathematical function which takes a set of inputs, adds weights and biases separately to each input and passes it through a non-linear function to produce the output. This output is carried to the perceptron of the next layer through a connection link along with certain other characteristics

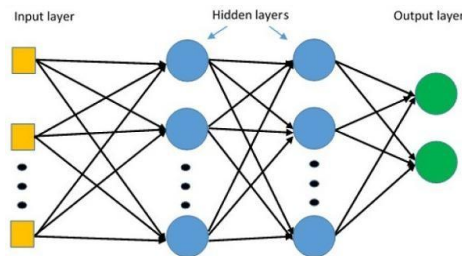
such as the activation signal, previous internal state, etc. depending on the type of neural network.

There are two types of perceptrons:

- Single-layer perceptron: They can only learn linearly-separable tasks. For example, linear binary classification problem
- Multi-layer perceptron: Also called a fully connected neural network. It contains two or more layers (input layer, hidden layer(s) and output layer), which helps bring out the non-linearity in the input data, thereby increasing processing power and helping the model learn better. For example, image recognition, stock analysis.



(a)



(b)

Figure 1.4.2: (a) A single-layer perceptron and (b) a multi-layer perceptron

1.5. Computer Vision

Computer vision is a subset of artificial intelligence which includes the methods designed to enable machines to process visual data such as images, videos and data from other visual inputs in order to derive meaningful information from them. These methods help machines gather the knowledge and learn the context to tell objects apart, understand the various components and features present in an image, and process sequences of slices of 3D images and videos.

Computer vision uses machine learning and deep learning models such as convolution neural networks (CNNs) to learn, analyse and interpret visual data like humans do. At a basic level, computer vision is mostly pattern recognition. Hence, the most common way to increase the efficiency of a computer vision algorithm is to feed it large amounts of labelled visual data and further use different methods and techniques to help the machine learn these patterns. Computer vision can further be classified into various domains based on its applications (figure 1.5.1.). For example, scene reconstruction, object detection, object recognition, event detection, motion estimation, 3D pose estimation, 3D scene modelling and image restoration to name a few.

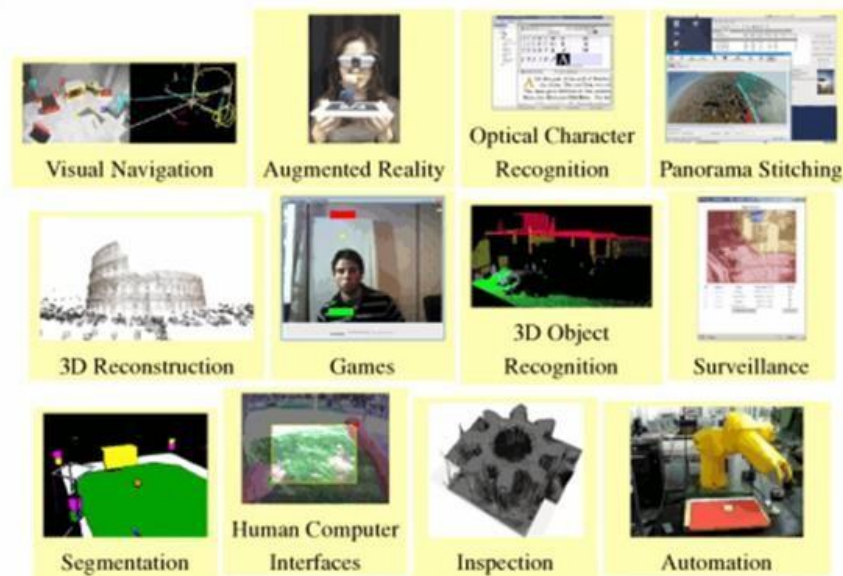


Figure 1.5.1.: Applications of Computer Vision

CHAPTER 2

LITERATURE REVIEW

In this section, we discuss the previous research that has been conducted in an effort to detect and identify aggressive driving behaviour. The information present in this section includes advancements in machine learning algorithms, robust neural network architectures, combinations of various tools used to enhance the performance of these algorithms and networks, and a background on the design and formulation of the neural network architectures used in our study.

Two factors are considered while studying aggressive driving behaviour [4]— the relative risk, i.e., the proportion of aggressive to non-aggressive instances and the odds ratio, i.e., the proportion of aggressive behaviour identified in one group to another.

2.1. DriftNet

Several attempts have been made before to identify suspicious and anomalous driving from videos. Different neural net architectures and methodologies used in anomaly detection and real time object detection include YOLO v3, TrackletNet tracker, F-RCNN, ResNet-50, GANs to handle difference in domains, Mobile inverted residual bottleneck blocks in 3D CNNs, DenseNet, LSTM and C3D [5].

The study, having compared C3D (parameters initialized in initial convolution layer), ConvLSTM (pre-trained on AlexNet as backbone) and DenseNet (bottleneck blocks, multiple dense layers and sample downed feature maps) (figure 2.1.1) with the parameters in Table 2.1.1, concluded that the 3D-DriftNet (DenseNet architecture) has the highest training and validation accuracy of 96.87% and 77.5% respectively.

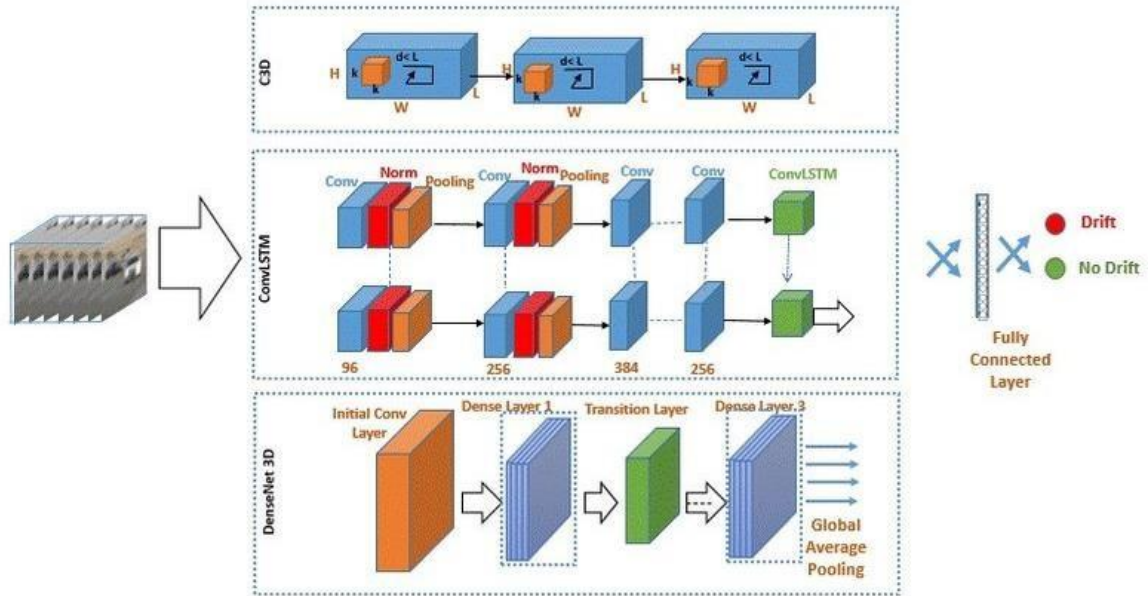


Figure 2.1.1: Comparison of three architectures
(a) C3D, (b) ConvLSTM and (c) DriftNet

Table 2.1.1: Parameters of DriftNet

Batch size	32
Learning rate	$10^{-2} - 10^{-4}$
Momentum	0.5
Decay	10^{-7}
Batch Normalization Decay	0.997
Resolution	112x112

2.2. Previous frameworks

Another study [6] classifies human driver inattentive and aggressive driving behaviour into two broad categories— habitual aggressiveness (intentional) and occasional aggressiveness (non-intentional). The detection framework was designed with the objective to incorporate it in technologies to aid people in behaviour correction. The framework was created by applying and comparing the following generative models to labelled datasets— multi-stream LSTM, LSTM + fully convolutional network, RNN+LSTM and Joint time series modelling (JTSM); along with using Gaussian mixture models to cluster and obtain the most distinctive driving styles and maximal information coefficient to compare and evaluate feature maps.

2.3. ResNets

Residual learning was first introduced [7] to solve the problems of high error rates, vanishing and exploding gradients that arise when deep neural networks are scaled depth-wise. The method uses the concept of skip connections/identity mapping/residual connections to ensure smoother gradient flow. There are two types of skip connections:

- Short: along consecutive convolution layers where input dimensions do not change. For e.g., ResNet, DenseNet.
- Long: used in encoder-decoder architectures (symmetrical) where spatial dimensionality decreases in encoder and increases in the decoder. For e.g., U-Nets.

Both these types enable feature reusability, by introducing activations of earlier layers via skip connections, and smoothens the loss landscape.

There are several variants and interpretations of ResNets that have since been introduced. Variations in terms of the number of layers, using an ensemble of ResNets as a single ResNet, increasing/decreasing the number of skip connections, and introducing concepts such as Stochastic Depth to enhance the performance of these networks, are a few examples.

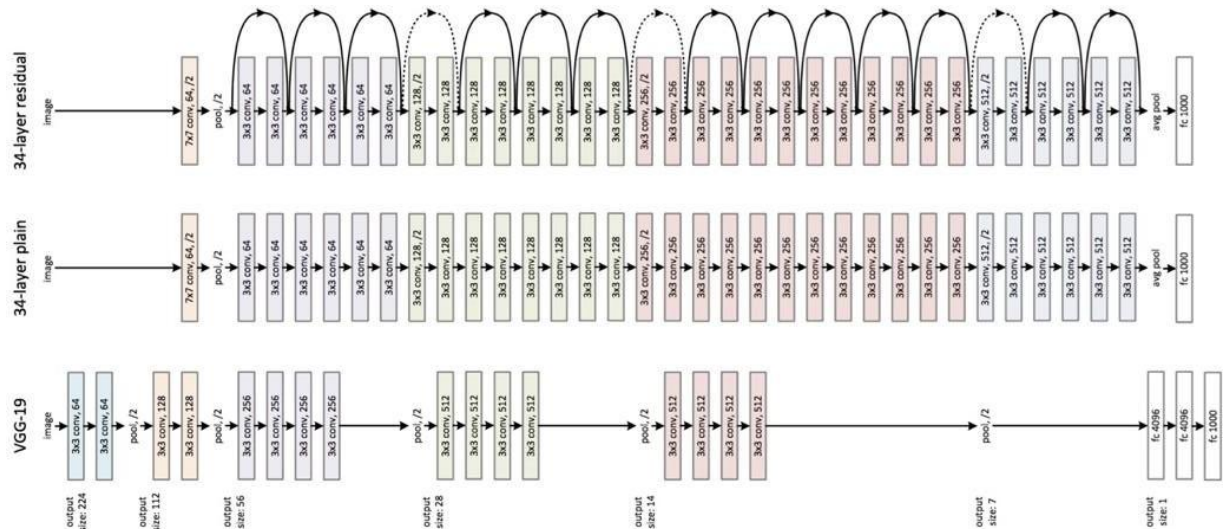


Figure 2.3.1.: Figure showing three different ImageNet-scaled network architectures—(a) VGG-19, (b) plain network with 34 parameter layers and (c) ResNet with 34 parameter layers

2.4. EfficientNets

EfficientNets were first introduced [8] to overcome the limitations of the manual hit and try method to rescale deep learning models to achieve higher accuracy. The base model (EfficientNet-B0) uses a compound coefficient (ϕ) to scale depth (D) and width (W) of the network (figure 2.4.1.), and the resolution (R) of the input images in a principal way:

depth: $d = \alpha^\varphi$

width: $w = \beta^\varphi$

resolution: $r = \gamma^\varphi$

where, $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ and,

$$\alpha \geq 1, \beta \geq 1, \gamma \geq 1 \quad (2.4.1)$$

where α, β, γ are constants that can be determined by a small grid search.

ϕ is fixed at 1 and increased linearly, forming EfficientNets B1 to B7. The researchers have also established a controlled trade-off between the accuracy and FLOPS (floating-point operations per second).

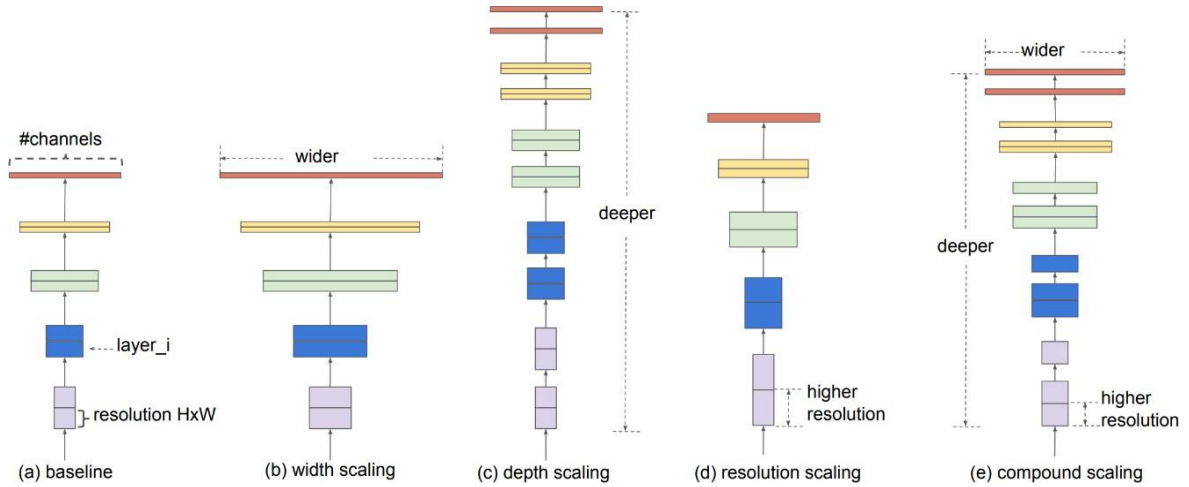


Figure 2.4.1: Scaling of a baseline model (a) using conventional methods—width (b), depth (c) and resolution of input images (d) and the proposed compound scaling of EfficientNet across all three dimensions (e).

The base model, upon training on 8 different image datasets with the parameters given in Table 2.4.1, achieved an 8.4x-9.6x reduction in trainable parameters, 16x reduction in FLOPS and 84.3% top-1 accuracy. Out of the 8 datasets, EfficientNets performed better on 5 of them, as compared to the existing models' best reported results. The EfficientNet-B7, being the most efficient of the 8 models, is also compared with the closest best performer (GPipe) and is found to be more accurate, smaller, and computationally cheaper.

- Scaling variables given by: $D=\alpha^\phi$, $W=\beta^\phi$, $R=\gamma^\phi$, where search space is limited to: $\alpha.\beta^2.\gamma^2 \approx 2$
- Best yield: $\alpha=1.2$, $\beta=1.1$, $\gamma=1.15$

Table 2.4.1: Parameters of EfficientNet

Dropout	0.2-0.5 (B0-B7)
Learning Rate	0.256 with a decay of 0.97
Momentum	0.9
Weight Decay	10-5
Batch Normalization Momentum	0.9
Optimizer	RMSProp
Activation	SiLU

2.5. RandAugment

Data augmentation can significantly improve the model's capability to generalize by reducing overfitting during training. It can also be used to address the limitations of size and variety of datasets. RandAugment [9] introduces a vastly simplified search space for data augmentation containing two interpretable hyperparameters (N= number of transformations and M= the magnitude of transformation) (figure 2.5.1), which can be employed to tailor the augmentation policy to a model and dataset by folding the parameters governing the policy into the hyperparameters of the model. In the pool of K transformations, each transformation has equal probability of being applied. A linear integer scale is employed to determine the extent of augmentation distortion (M). Large values of N and M result in increased regularization strength. For ImageNet-scale networks, the preferred values of N and M specified in the research are 2 and 9 respectively.

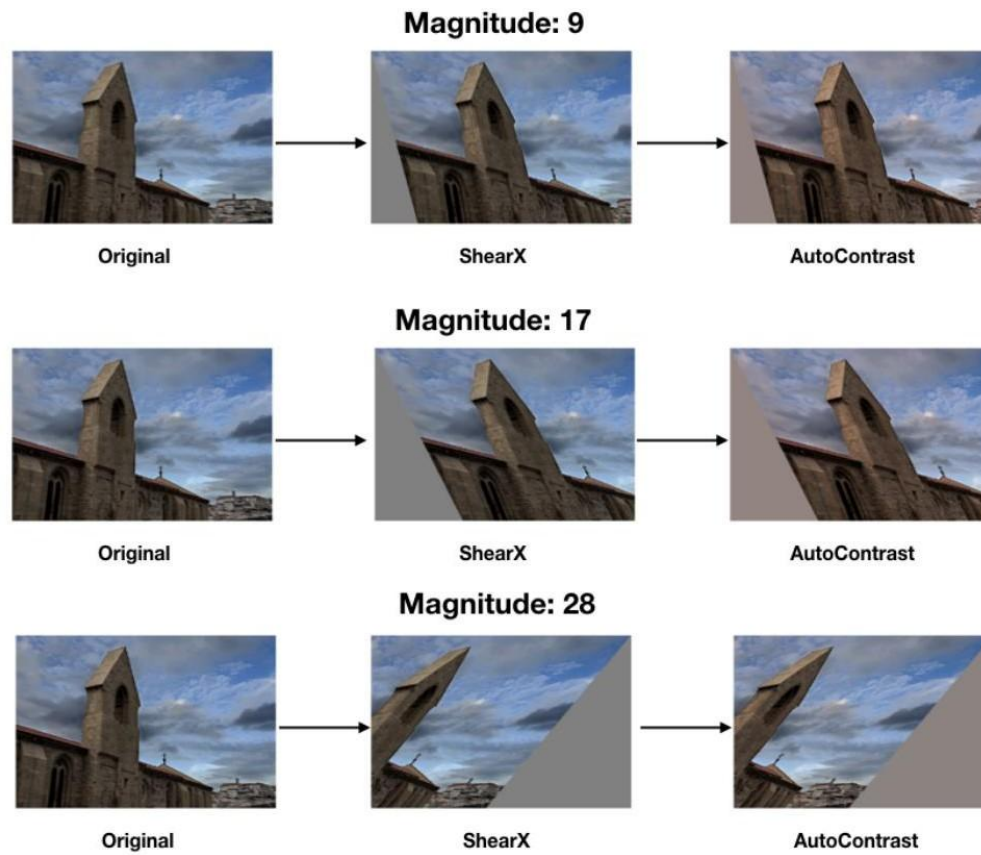


Figure 2.5.1.: Examples of images augmented by RandAugment with number of transformations(N)=2 and the distortion magnitude(M)=9 (a), 17 (b), and 28 (c) for ResNet-50, EfficientNet-B5 and EfficeintNet-B7 respectively.

2.6. Hausdorff distance

Hausdorff distance is used to measure the distance between two subsets of a metric space. It is most commonly used as a performance metric for comparing segmented images with the original dataset. A small magnitude indicates higher precision of segmented images. In a recent study [10, 11] detecting crowd group structures, Hausdorff distance was used as a measure of inter-group closeness and locomotion similarity that takes into consideration the variables of proximity and velocity, having shown to be used for crowd movement and trajectory analysis

[12] prior to it.

Symmetric Hausdorff distance between group A and B is

$$H(A, B) = \frac{h(A, B) + h(B, A)}{2}, \text{ where} \quad (2.6.1)$$

$$h(A, B) = \frac{\sum_{i=1}^{|A|} \sum_{j=1}^{\lceil |B|/2 \rceil} \text{dij}}{|A| \times \lceil |B|/2 \rceil} \quad (2.6.2)$$

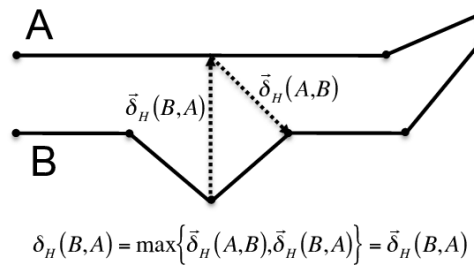


Figure 2.6.1: (a) Image depicting the calculation of Hausdorff distance between two polygon curves

CHAPTER 3

PROPOSED METHODOLOGY

In this section, we provide the details on the dataset used, the proposed architecture for the identification of aggressive driving behaviour and the algorithms used to detect the set of cars involved in the incident. First of all, we created a dataset containing video clips of both aggressive and non-aggressive driving instances. Secondly, using a combination of highly complex and efficient pre-trained neural networks, we created the feature extractor, along with a simple recurrent neural network architecture for the classification of the video clips. Finally, using frame differencing, thresholding, and spatial cluster distance of the segmented cars, we detect and annotate the set of frames where the cars involved in the incident are present.

3.1. Dataset

There are several sources of datasets containing traffic footage, ranging from CCTVs to aerial shots of single and multi-laned roads with traffic directed in both the directions. Certain videos contain cars at intersections, traffic lights, common junctions, and L bends diversions. A few examples include:

- Roads, cars and people video: published in 2018. This dataset consists of videos of roads, cars, people, road conditions and traffic shot at thirty two different locations in Vancouver, with four cameras recording traffic at intersections from either an angle of 45 degrees or 90 degrees.
- Highway traffic videos dataset: this dataset contains videos of traffic on a highway in Seattle, Washington, with categories labelled as light, medium and heavy traffic.
- MIT traffic used for conducting research on crowded scenes and analysis of activities. It has a traffic video that is 90 minutes long, shot by a stationary camera and is divided into 20 clips.

- The Car Accident Detection and Prediction dataset (CADP) with a fraction of the video clips having full spatio-temporal annotations.
- The Video Image Retrieval and Analysis Tool (VIRAT) video dataset containing video clips of surveillance from several public CCTV cameras, a fraction of which form traffic and parking-lot surveillance videos. It is used for event recognition tasks from these surveillance video clips.

However, none of the datasets listed above contain two distinct classes of videos labelled as ‘aggressive’ and ‘non-aggressive’, with videos shot from cameras placed at similar angles to the roads. In addition to this, the videos also contain other elements such as a foreign object intervention, pedestrian involvement in accidents at junctions, and top views of highways.

Due to the lack of a compiled and annotated large-scale dataset containing video footage of traffic shot from a uniform pivoted point-of-view, we created a dataset containing 30 video clips for both the aggressive and non-aggressive classes. The aggressive class contains short clips of cars overtaking from the wrong side, switching lanes in a way that poses harm to other vehicles, close L-bend diversions and passing on the shoulder. The non-aggressive class contains short video clips of cars whose drivers are shown to abide by traffic rules that govern the situations stated earlier. The dataset was made uniform to the greatest extent by taking into consideration factors such as low lighting, scenery, number of lanes, direction of the movement of cars, and the amount of traffic on these roads.

The videos were downloaded from YouTube and Stock video sites, cropped for obtaining a moderately uniform view of the roads and traffic, carefully analysed for the presence and occurrence of rash driving defined earlier, and the videos containing such incidents were trimmed and added to the directory labelled as ‘aggressive’. In order to balance the dataset to a maximum possible state both quantitatively and qualitatively, video clips with almost the same running time, containing incidents of regular and safe driving were trimmed and added to the directory labelled as ‘non-aggressive’. Figure 3.1.1. shows a set of frames extracted from the video clips in the datasets.



Figure 3.1.1.: Sample Frames from the Dataset

The paths to these video files and their labels are then stored in a data frame, making it easier for further pre-processing and encoding. To handle the problem of a small-scale and unbalanced dataset, RandAugment [9] was used to transform the frames and increase the size of the dataset. The number of distortions (N) and the magnitude (M) used were 2 and 9 respectively. A total of 1323 frames were extracted for both the classes. The frames were resized to a size of 260x260 during the extraction. However, the number and the order of the colour channels remained the same. Additionally, no simple augmentation of the individual frames were carried out during frame extraction. RandAugment was performed on the frames after they were extracted from

the individual video clips. Figures 3.1.2 and 3.1.3 show the first five records of the data frames containing the paths to both the training and test datasets, along with their labels.

	path	label
0	/content/drive/MyDrive/major project/videos/tr...	aggressive
1	/content/drive/MyDrive/major project/videos/tr...	aggressive
2	/content/drive/MyDrive/major project/videos/tr...	aggressive
3	/content/drive/MyDrive/major project/videos/tr...	aggressive
4	/content/drive/MyDrive/major project/videos/tr...	aggressive

Figure 3.1.2.: First five records of the data frame containing the paths to the videos in the training set along with their labels

	path	label
0	/content/drive/MyDrive/major project/videos/te...	aggressive
1	/content/drive/MyDrive/major project/videos/te...	aggressive
2	/content/drive/MyDrive/major project/videos/te...	aggressive
3	/content/drive/MyDrive/major project/videos/te...	aggressive
4	/content/drive/MyDrive/major project/videos/te...	aggressive

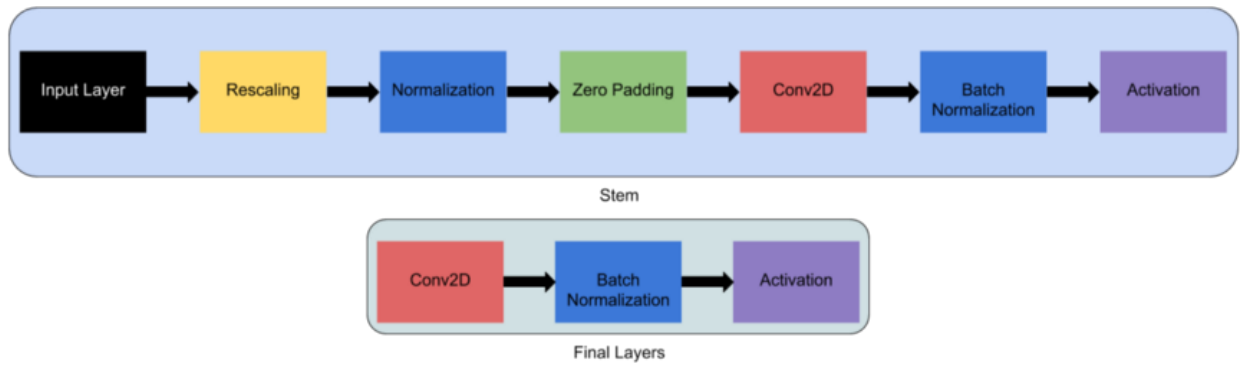
Figure 3.1.3.: First five records of the data frame containing the paths to the videos in the testing set along with their labels

3.2. Feature Extractor

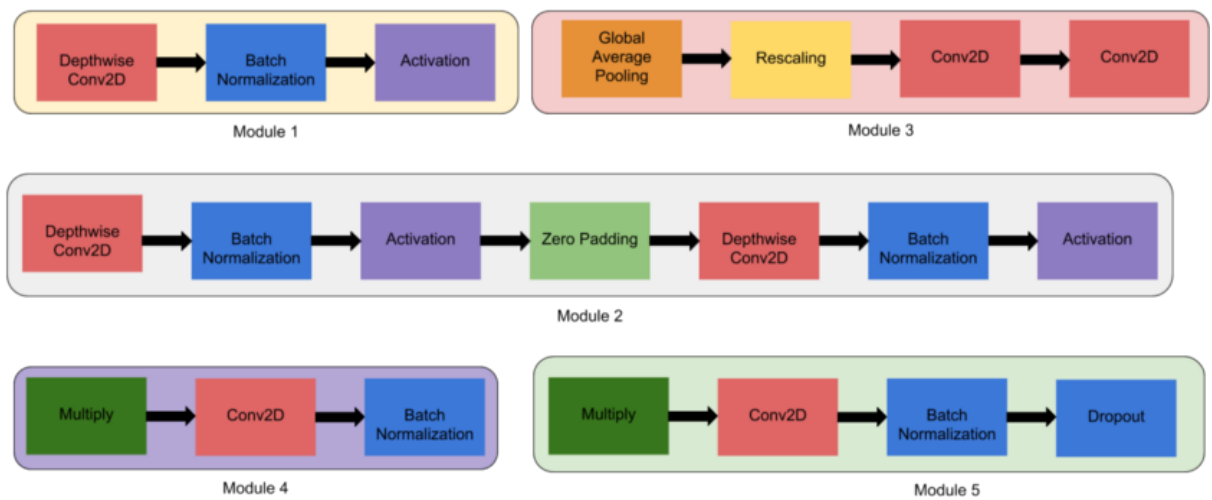
A convolution neural network (CNN) is a type of deep feed-forward neural network that is used to process data with spatial and temporal dependencies. They are most commonly used for image recognition, classification and detection tasks. Convolution is the mathematical function, a special form of a linear operation, which produces a function as a complex product of two different functions, that in-turn determines how the shape of one function is modified by the other. In terms of image processing, two images (in the form of tensors) are multiplied, resulting in an output that is used to extract the features of the image. The various layers of a CNN are:

- The Convolution Layer: This layer performs the convolution operation between the input images and a kernel of size defined in the layer, resulting in the extracted features. It usually forms the first layer of any CNN, following the input layer that defines the resolution of the input images. The kernel sizes usually range from 1x1, 3x3, 5x5, 7x7, etc.
- The Pooling Layer: This layer decreases the number of features in the output of the convolution layer by decreasing the number of connections between the layers and independently computing each feature map. The two most commonly used pooling techniques are discussed later in this section.
- The Fully Connected Layer: It consists of the flatten and the dense layers, where the output of the convolution and pooling layers are flattened and passed through several dense layers where the linear/non-linear operations take place. They contain weights and biases which are tuned during training. This usually forms the final classification part of a CNN.
- Dropout layers: These layers are used to induce dropouts by randomly silencing a selected number of neurons at a time during training. This method helps in reducing overfitting by decreasing the size and complexity of the model. For eg., a dropout value of 0.25 results in 25% of the neurons to be randomly “dropped” during training.

A combination of two state-of-the-art convolution neural networks pre-trained on the ImageNet dataset, the EfficientNet-B7 and the ResNet-50, were used to extract features from the frames of the video clips. Since the EfficientNet-B7 has proved to be more accurate and efficient as compared to the ResNet-50, it is used in the beginning to extract the initial, more important features. The architectural details of the EfficientNet-B7 and the ResNet-50 are depicted in figure 3.2.1 and 3.2.2 respectively.



(a)



(b)

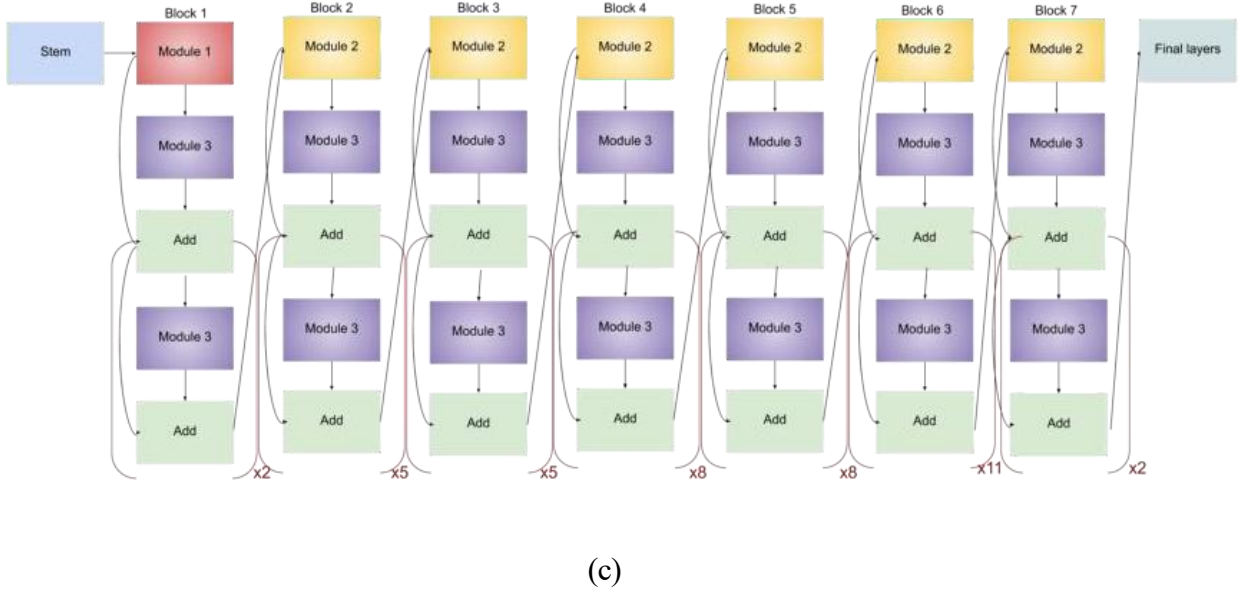


Figure 3.2.1.: Architectural details of the EfficientNet-B7 (a) defining the layers of the stem and the final blocks (b) defining the layers of the modules used in the final EfficientNet-B7 network and (c) defining the final architectural design of the EfficientNet-B7 with the modules and blocks of layers defined earlier

The top layer of the EfficientNet is excluded and hence, we define an input layer with the input size as the resolution of the processed frames, i.e., (260 x 260 x 3). In this case, since we are using the pre-trained network, the weights of the network that was trained on the ImageNet is loaded. There is no pooling defined, thereby receiving the exact number of the extracted features as output. The classifier activation is set to ‘softmax’, the default function.

The ResNet-50 is a ResNet model with fifty layers in total—48 convolution layers, one MaxPool and one AveragePool layer. As opposed to the convention of having skip-connections skipping two layers as mentioned in the research [7], the ResNet-50 contains skip-connections that skip three convolution layers. Additionally, it has 3.8×10^9 floating point operations.

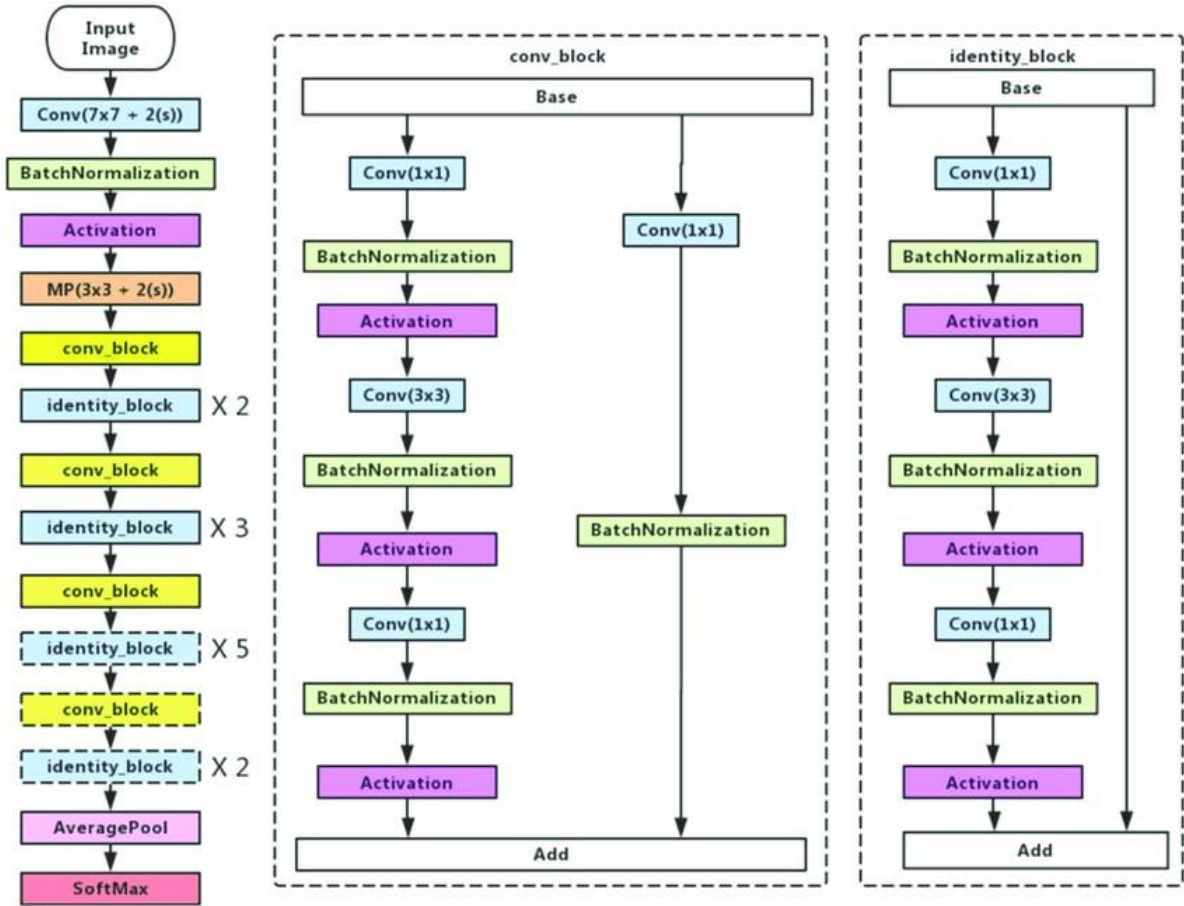


Figure 3.2.2.: Architectural details of the ResNet-50

In our project, the ResNet-50 model is defined without the top layer, since we have different input shapes as compared to the original model. The input shape is calculated using the formulas stated below

$$\text{Output Height} = \frac{\text{Input height} + \text{padding height top} + \text{padding height bottom} - \text{kernel height}}{\text{stride height}} + 1 \quad (3.2.1)$$

$$\text{Output Width} = \frac{\text{Output width} + \text{padding width right} + \text{padding width left} - \text{kernel width}}{\text{stride width}} + 1 \quad (3.2.2)$$

Furthermore, the ResNet-50 model, like the EfficientNet, is loaded with the ImageNet weights. Prior to sending the images and features as the inputs to the EfficientNet and the ResNet respectively, the inputs must be pre-processed. Thus, layers to pre-process the inputs are defined and introduced before the models. Since the pre-processing layers and methods are included in the main model architecture for the EfficientNet as the Rescaling layer, the `preprocess_input` layer for this model is essentially a pass-through function. However, for the ResNet, the `preprocess_input` layer converts the RGB images to BGR and performs zero centre normalization on the colour channels without scaling.

Pooling is used to reduce the dimensions of the features produced by the network. By reducing the dimensions and the features, it reduces the number of parameters to be learnt by the model during training, thereby reducing the computation cost of training the network, and increases its efficiency. There are two types of pooling that can be performed on the output of features:

- **Max Pooling:** in this type of pooling, it returns the maximum value of the pixels that are covered by the kernel. Since the maximum value among a set of pixels is towards the lighter end of the colour spectrum, Max Pooling selects the brighter and lighter-coloured pixels of the image. This can be used for tasks where the background of the object to be segmented is dark or if this object needs to be distinctively identified.
- **Average Pooling:** in this type of pooling, it computes and returns the average of the pixel values that are covered by the kernel. In general, Average Pooling smoothens out the image, and prevents the sharp features of the images such as edges, lines, and grains from being retained.

The extracted features of the EfficientNet are reshaped to an appropriate size containing three colour channels, thus having the input shape compatible with that of the ResNet and simultaneously preventing the loss or transformation of the extracted features by using additional Convolution or Pooling layers. Both Max-Pooling and Average Pooling are first applied across the extracted features of the ResNet to obtain a total of 2048 features for each frame. The pooling producing the higher score of accuracy during training is then retained.

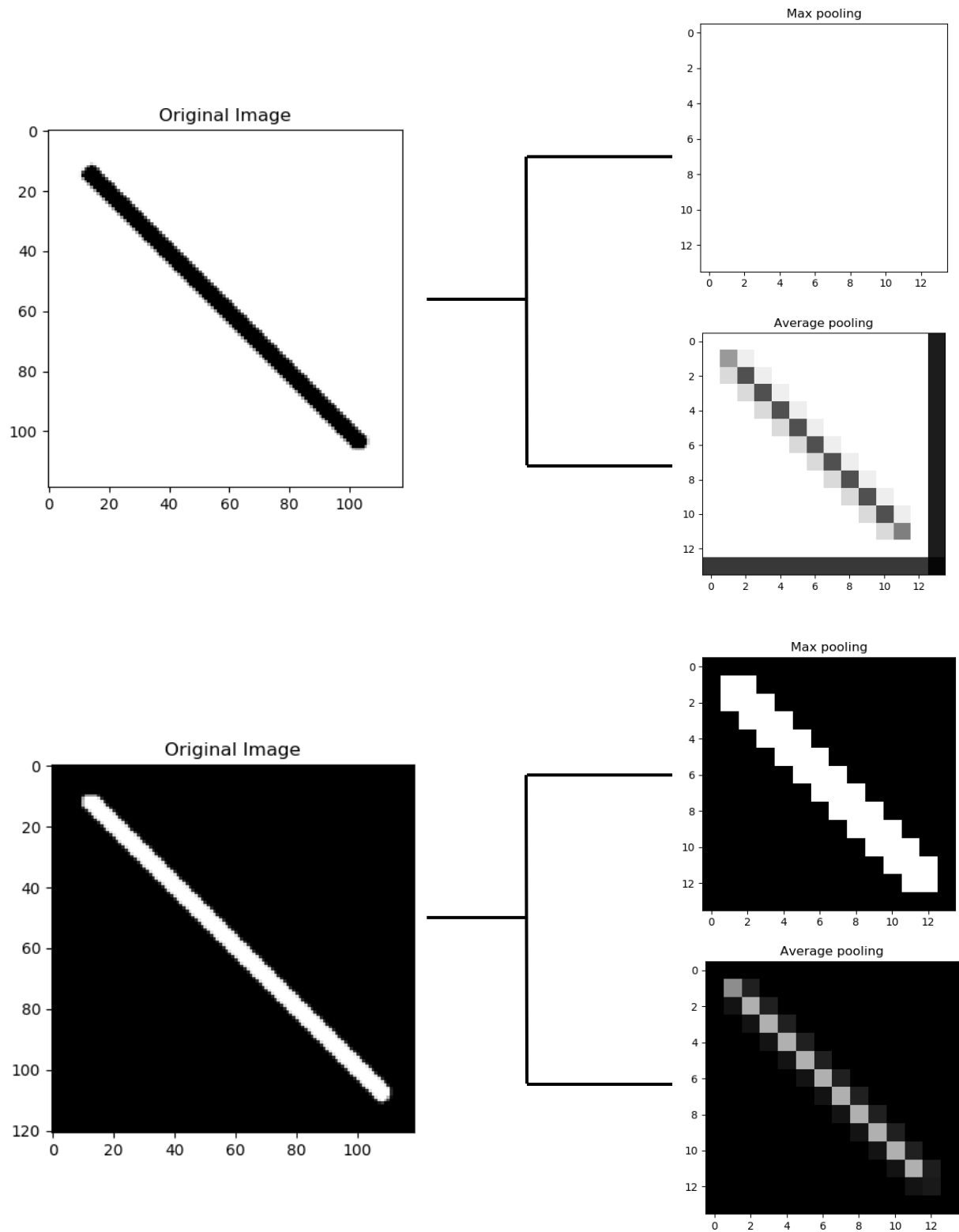


Figure 3.2.3.: Differences between Max Pooling and Average Pooling

The extracted features of the frames for each video are stored in an array and padded with zeros till a maximum sequence count is reached. This maximum sequence count, i.e., the maximum frames per video allowed as input, is defined to limit the length of the sequence of frames given as input, hence keeping a check on the computation cost and time. A Boolean timestamp array is also initialized for each video. This array, containing zeroes and ones, defines those elements of the array of features to be considered (indicated by ones) by the network while training, thus helping the network distinguish between padded values and features of the frames extracted from the respective videos.

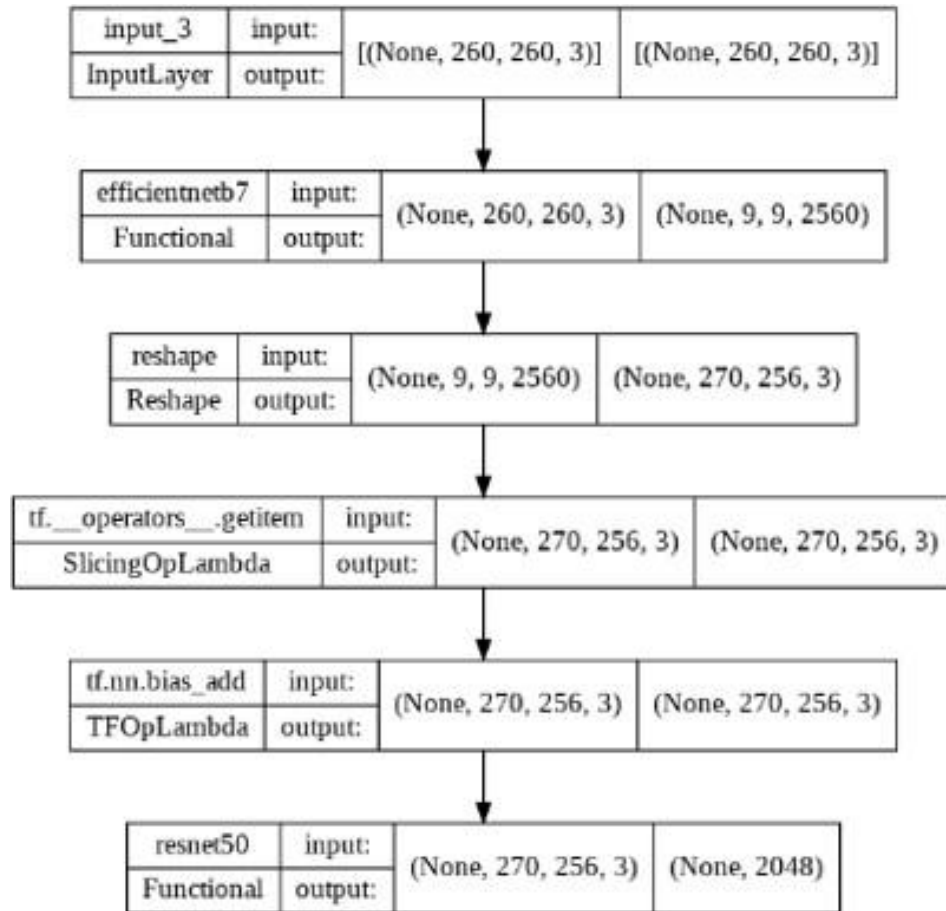
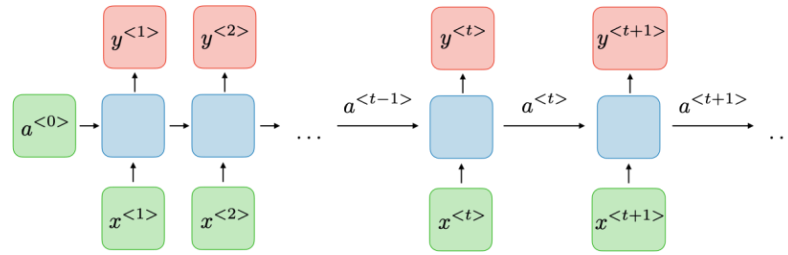


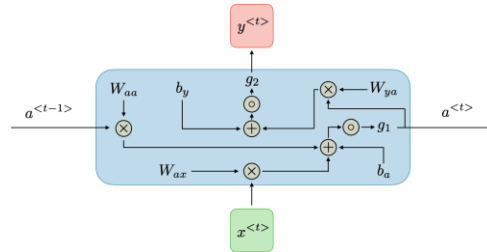
Figure 3.2.4.: Proposed Architecture of the feature extractor CNN

3.3. Classifier

A recurrent neural network is a form of a deep neural network with recurrent connections i.e., where the output of a layer is saved and is fed back as the input to this layer in order to compute and predict the output of this particular layer. It was mainly devised as a method to handle sequential data, where the temporal or sequential attributes of the individual elements of the dataset are correlated with the output of the model.



(a)



(b)

Figure 3.3.1.: Architectural details of (a) a traditional RNN and (b) the individual blocks of recurrent connections

For a given time ‘t’, the activation a^t and the output y^t are given as:

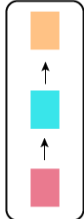
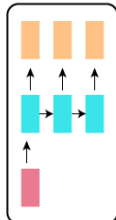
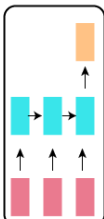
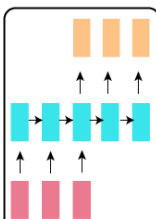
$$a^t = g_1 (W_{aa}a^{(t-1)} + W_{ax}x^{(t)} + b_a) \quad (3.3.1)$$

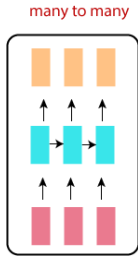
$$y^t = g_2 (W_{ya}a^{(t)} + b_y) \quad (3.3.2)$$

where W_{ax} , W_{aa} , W_{ay} , b_a , b_y are the coefficients (weights and biases respectively) and g_1 and g_2 are the activation functions.

RNNs can be further classified based on the problem statement they are applied to:

Table 3.3.1.: Different types of RNNs

One to one		Simple neural networks dealing with fixed sizes of inputs and outputs. Eg. Image classification.
One to many		Fixed input size, producing sequential data as output. Eg. Caption generation.
Many to one		It takes sequential data as input and produces a single output. Eg. Classification tasks
Unidirectional many to many		It takes sequential data as input and produces a corresponding sequential data as output. Eg. Language translation.

Bidirectional many to many		Similar to the unidirectional many to many RNN architecture, but the output is based on the individual elements of the input, i.e., they are synced. Eg. Frame labelling in a video.
----------------------------	---	--

For this project, a recurrent neural network, consisting of two SimpleRNN layers with 64 units each, a dense layer and an output layer is designed as the classifier that identifies patterns of aggressive driving. Table 3.3.2. provides details on the different attributes of the SimpleRNN() function taken as input arguments during initialization. These also contain the parameters that are tuned during the process to enhance the performance of the model.

Table 3.3.2.: Attributes of the SimpleRNN() function [13]

Attributes	Description
Units	Positive integer, dimensionality of the output space.
Activation	Activation function to use. Default: hyperbolic tangent (tanh). If you pass None, no activation is applied (ie. "linear" activation: $a(x) = x$).
use_bias	Boolean, (default True), whether the layer uses a bias vector.
kernel_initializer	Initializer for the kernel weights matrix, used for the linear transformation of the

	inputs. Default: glorot_uniform.
recurrent_initializer	Initializer for the recurrent_kernel weights matrix, used for the linear transformation of the recurrent state. Default: orthogonal.
bias_initializer	Initializer for the bias vector. Default: zeros.
kernel_regularizer	Regularizer function applied to the kernel weights matrix. Default: None.
recurrent_regularizer	Regularizer function applied to the recurrent_kernel weights matrix. Default: None.
bias_regularizer	Regularizer function applied to the bias vector. Default: None.
activity_regularizer	Regularizer function applied to the output of the layer (its "activation"). Default: None.
kernel_constraint	Constraint function applied to the kernel weights matrix. Default: None.
recurrent_constraint	Constraint function applied to the recurrent_kernel weights matrix. Default: None.
bias_constraint	Constraint function applied to the bias vector. Default: None.
Dropout	Float between 0 and 1. Fraction of the units to drop for the linear transformation

	of the inputs. Default: 0.
recurrent_dropout	Float between 0 and 1. Fraction of the units to drop for the linear transformation of the recurrent state. Default: 0.

An activation function is a non-linear function performed on the weighed and biased inputs of a perceptron, to obtain the output to be sent to the next layer. Non-linearity helps the model learn better by learning complex correlations and hidden features of the data and deviating from the ideal conditions and attributes. The two types of activation functions recommended for use in an RNN are:

- Sigmoid: Also known as the logistic activation function, it returns values between 0 and 1. The function, being smoother, allows for ease of classifying elements of a binary classification problem. However, a high negative input produces a value close to 0, thereby reducing the speed with which weights are updated during back-propagation.

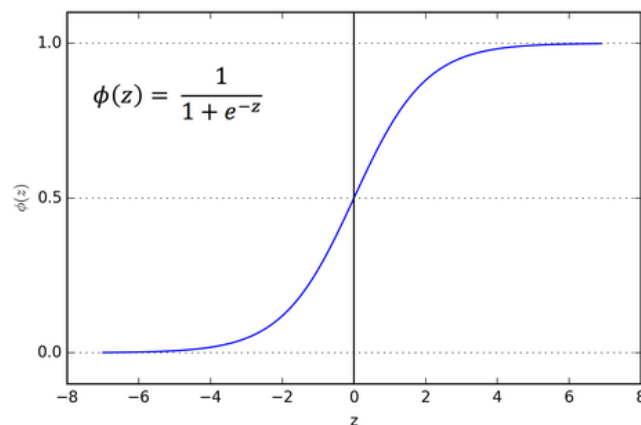


Figure 3.3.2.: Sigmoid function

- **Tanh:** Also known as a hyperbolic tangent activation function. This function returns values between -1 to 1. The negative inputs are mapped in the negative half of the range. Though its outputs are zero centre normalised and the convergence is usually faster, the error surface can be flat at the origin, hence initializing with very small weights should be avoided.

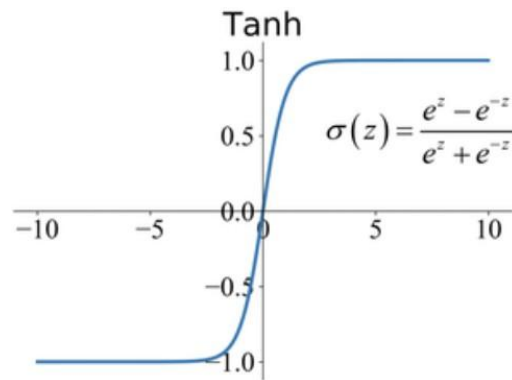
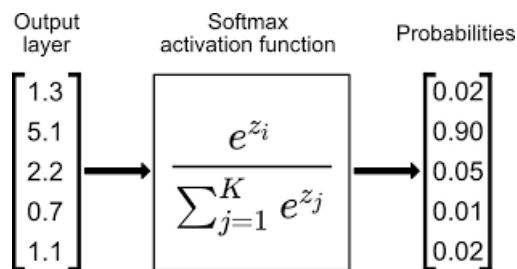


Figure 3.3.3.: Tanh function

- **Softmax:** This activation function, usually used in the final layer of neural networks designed for classification, returns values between 0 and 1 (the probabilities of their occurrence in different classes), where all the inputs sum up to 1.



$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

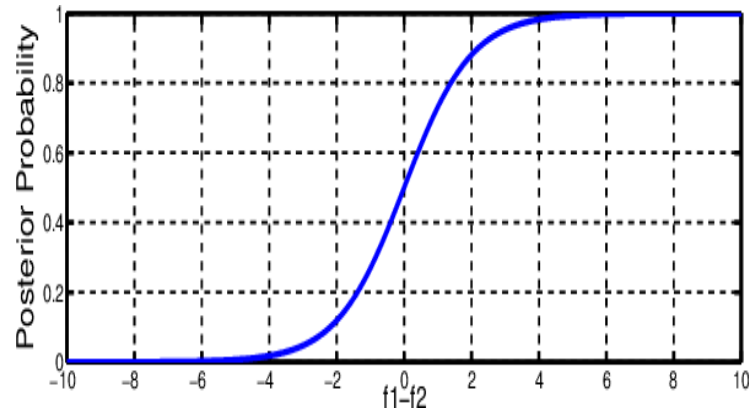


Figure 3.3.4.: Softmax function

Vanishing and exploding gradients are common problems faced while training deep neural networks. During backpropagation, the gradients can either get extremely small or large as the propagation algorithm progresses. This causes the weights in the initial layers to be left nearly unchanged or cause major changes respectively. In order to prevent the occurrence of vanishing and exploding gradients, proper weight initialization for the different layers is required. The various weight initialisers used in the RNN definition are:

- Glorot Normal: Also called a Xavier normal initializer, it defines the initialization as a truncated normal distribution with

mean = 0 and,

$$\text{standard deviation} = \sqrt{\frac{2}{\text{fan}_{\text{in}} + \text{fan}_{\text{out}}}} \quad (3.3.3)$$

where, fan_{in} and fan_{out} are the number of input and output units in the weight tensor.

- Glorot Uniform: Also called a Xavier uniform distribution, it defines the initialization

as a uniform distribution within a range $[-x_a, x_a]$, where,

$$x_a = \sqrt{\frac{6}{f_{an_{in}} + f_{an_{out}}}} \quad (3.3.4)$$

where, $f_{an_{in}}$ and $f_{an_{out}}$ are the number of input and output units in the weight tensor.

- LeCun Normal: It defines the initialization as a truncated normal distribution with

mean = 0 and,

$$\text{standard deviation} = \sqrt{\frac{1}{f_{an_{in}}}} \quad (3.3.5)$$

where, $f_{an_{in}}$ is the number of input units in the weight tensor.

- LeCun Uniform: it defines the initialization as a uniform distribution within a range $[-l_e, l_e]$, where,

$$l_e = \sqrt{\frac{3}{f_{an_{in}}}} \quad (3.3.6)$$

where, $f_{an_{in}}$ is the number of input units in the weight tensor.

- Orthogonal: The initialization is defined as a random orthogonal matrix, i.e., the rows and columns are orthonormal vectors.
- Random: The initialization is defined as either a random normal distribution with the specified mean and standard deviation or a uniform distribution within the specified range.

Bias and variance are prediction errors that occur while training a machine learning model. Bias

can be defined as the difference between the average prediction of the model and the ground truth values. Whereas variance is the model's sensitivity to and variation in predictions for different portions of the training dataset. There is a trade-off between these two parameters when the model is required to simultaneously minimize both.

A model is said to be a good fit when it learns most of the patterns present in the dataset but not to the extent of including outliers and random data points, and hence is able to generalize efficiently by producing the right predictions for new unseen data. (figure 3.3.5)

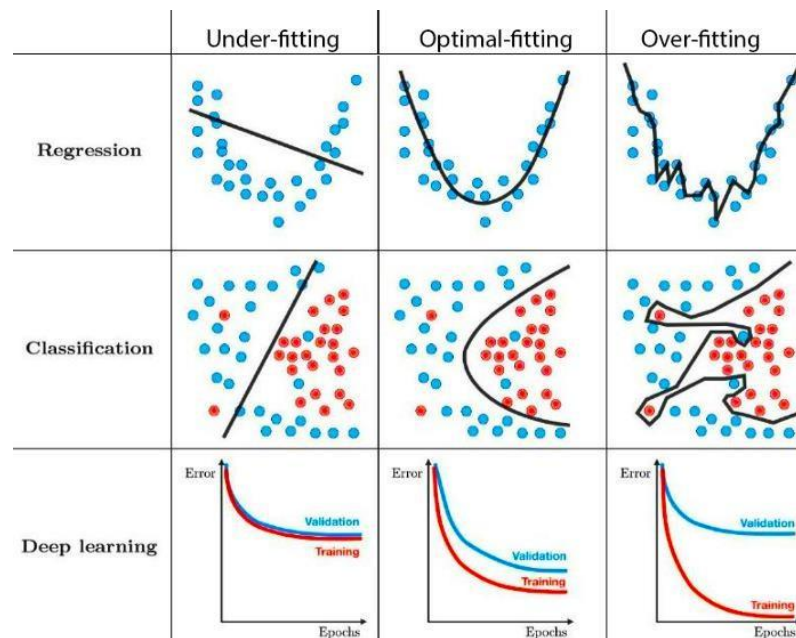


Figure 3.3.5.: Curves of predictions and true values for different classes of model performance

When a model is unable to learn the inherent patterns and correlations between the variables of a dataset, it is said to underfit. In this case, the model is not efficient and has to be enhanced or replaced with a more complex model that is appropriate for the task. On the other hand, allowing a model to learn the data to an extent where it learns a large number of patterns pertaining only to the training data, including erroneous (noise) and rare occurrences of data points, will cause the model to overfit. Such models perform exceptionally on the training set but fail to generalize and hence under-perform on the test and validation datasets. An underfitting model is said to

have high bias and low variance while an overfitting model is said to have high variance and low bias

Regularization is a technique used to add a penalty to the model as its complexity increases, in order to penalize the parameters, thereby helping it generalize and prevent it from overfitting. The two types of regularization methods used in the SimpleRNN layers are:

- L1 regularization: Also called as Lasso (Least Absolute Shrinkage and Selection Operator) regression, adds the absolute value of the magnitude of the coefficient as the penalty term to the loss function. This type of regularization shrinks the less important feature's coefficients to zero, thus acting as an implicit feature selection algorithm. The modified loss function is given as:

$$\sum_{i=1}^n (Y_i - \sum_{j=1}^p X_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p |\beta_j| \quad (3.3.7)$$

- L2 regularization: Also called as Ridge regression, which adds the squared magnitude of the coefficient as the penalty term to the loss function. This type of regularization helps prevent over-fitting of the model. The higher the value of the penalty, the greater is the decrease in magnitude of the coefficients. It thus reduces the complexity of the model by shrinking the coefficients. The loss function is given as:

$$\sum_{i=1}^n (Y_i - \sum_{j=1}^p X_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p \beta_j^2 \quad (3.3.8)$$

where in both the equations, the penalty term is denoted by λ and the first component depicts the loss (sum of squared residuals).

The array of sequences containing the extracted features for each video, along with their Boolean timestamp arrays form the inputs of this network. The Boolean timestamp arrays help the model in differentiating between the true values of the extracted features and padded sequences. Figure 3.3.7 shows the architecture followed in this project for the RNN classifier.

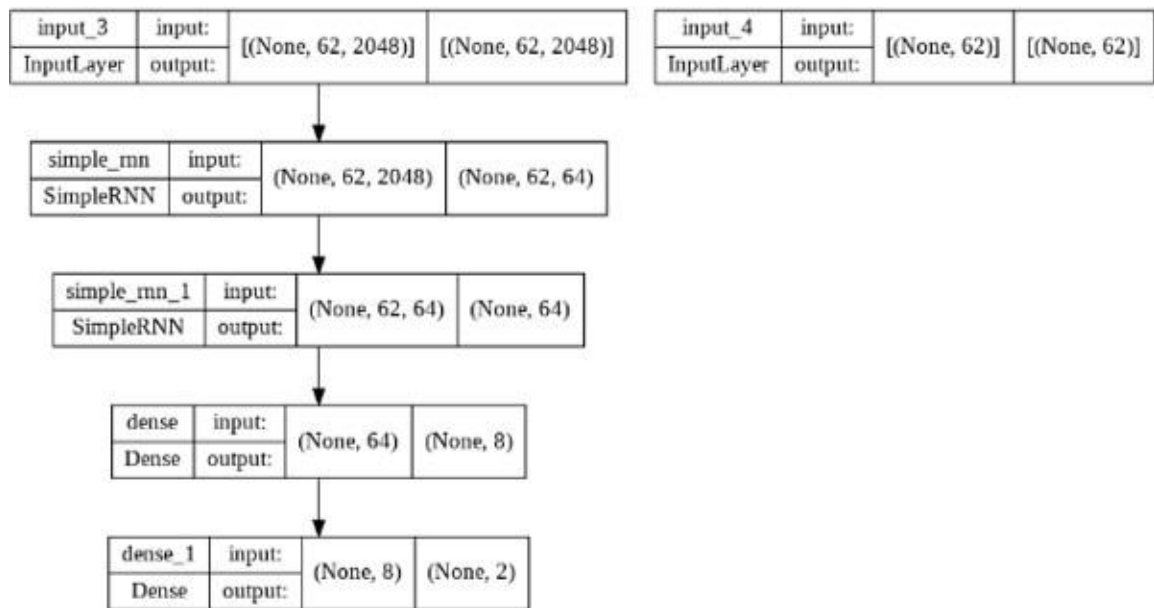


Figure 3.3.6.: Proposed Architecture of RNN Classifier

3.4. Detection

This module helps detect the individual vehicles involved in an incident of aggressive driving, To prevent the need for training another machine learning model or a neural network such as bounding box and coordinate point regressors, we have leveraged computer vision methods to segment and identify the relevant vehicles from the individual frames of a video that is classified by the model as ‘aggressive’.

The segmentation of vehicles on the roads and the detection of the those involved in aggressive driving is done with the help of computer vision tools. Upon being classified as ‘aggressive’, the frames of the video are extracted and pre-processed by reshaping the frames to a standard size and converting the RGB frames to greyscale. Binary thresholding is applied on the greyscale frames and all moving objects in the video are segmented using frame differencing. Figure 3.4.1. shows a sample set of frames obtained from the video after thresholding. The contours for these segmented objects are then obtained and the ones with significant total contour areas are considered for further processing. Self-intersecting and entirely overlapped contours are removed and the Hausdorff distances [13] between the contour in consideration and the other contours present in the frame are computed.

The directed Hausdorff distance H between two point sets A and B is the maximum of distances between each point $x \in A$ to its nearest neighbour $y \in B$. That is

$$H(A, B) = \max_{x \in A} \{\min_{y \in B} \|x, y\|\} \quad (3.4.1)$$

where $\|.,.\|$ is the Euclidean distance function.

The contours with distances greater than the mode times the area is used to obtain the final bounding box coordinates containing all such contours. In moving traffic, since it is not possible to identify a single vehicle that caused this incident, all the vehicles involved in the incident are detected. The subsequent frames are then annotated with the box.

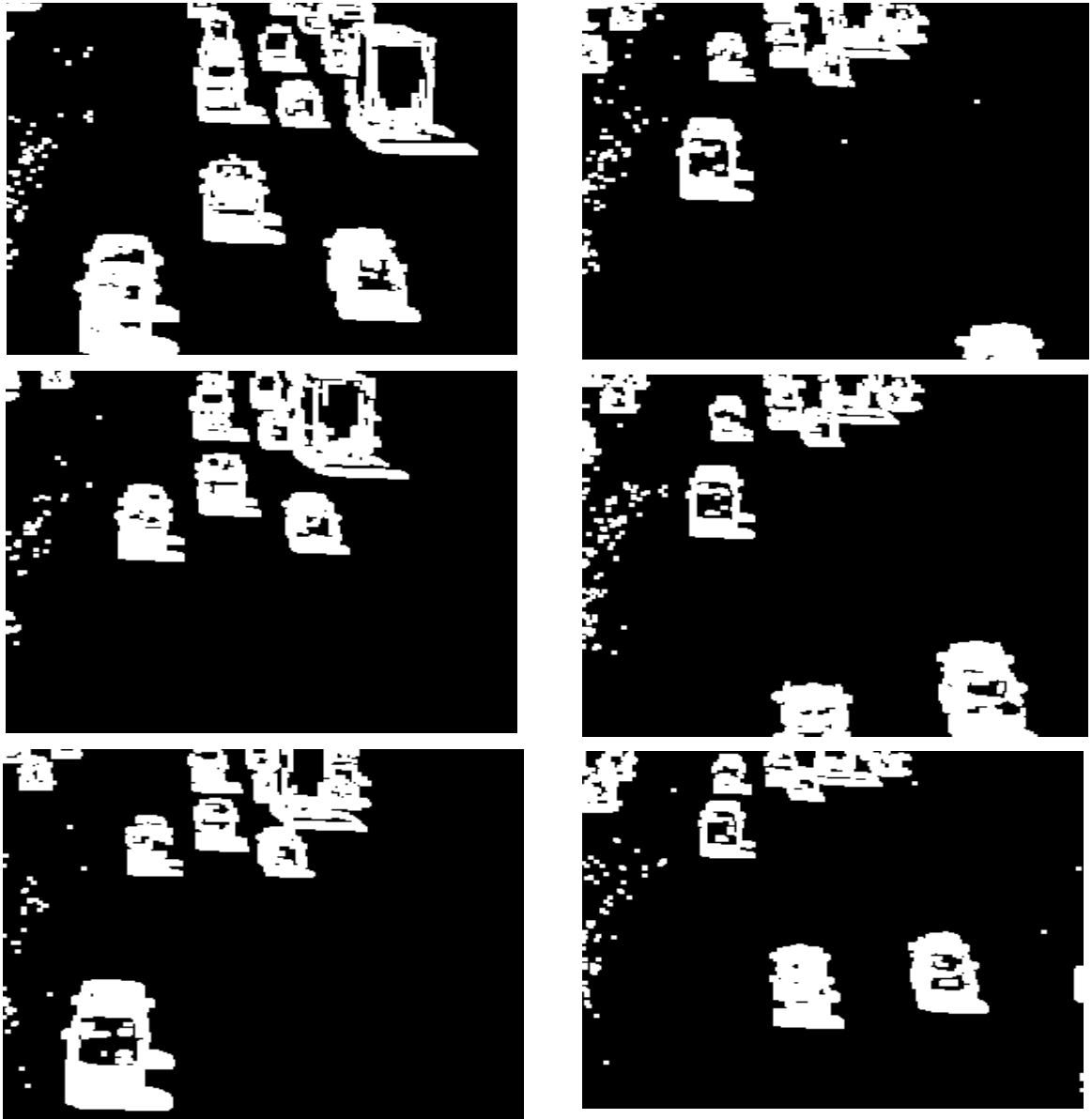


Figure 3.4.1.: Frames Post Thresholding and Dilation

3.5. Algorithm

1. Read .mp4 files to capture the frames and calculate the maximum frames per video (i.e., maximum sequence length).
2. Load and store the labels and encode them using the indices and their occurrence (string lookup).
3. Combine EfficientNet-B7 and ResNet50 to form the feature extractor and calculate the input and output shapes using the following formulas:

$$\text{Output height} = (\text{Input height} + \text{padding height top} + \text{padding height bottom} - \text{kernel height}) / (\text{stride height}) + 1$$

$$\text{Output width} = (\text{Output width} + \text{padding width right} + \text{padding width left} - \text{kernel width}) / (\text{stride width}) + 1$$

4. Extract the features and the boolean timestamp mask of the frames for all the videos. Pad features with zeros for videos having a frame count less than the maximum sequence length.
5. Construct a sequential model with SimpleRNN layers. Define checkpoint for storing weights of the best results and train the model on the extracted features and masks as input and the encoded labels as output.
6. For positive predictions of aggressive driving, obtain the frames and segment the vehicles using frame differencing and a specified threshold.
7. Check for overlapping and self-intersecting contours and obtain those with Hausdorff distances greater than the specified minimum value.

8. Obtain the bounding box coordinates for these contours and their corresponding frames as the outputs.

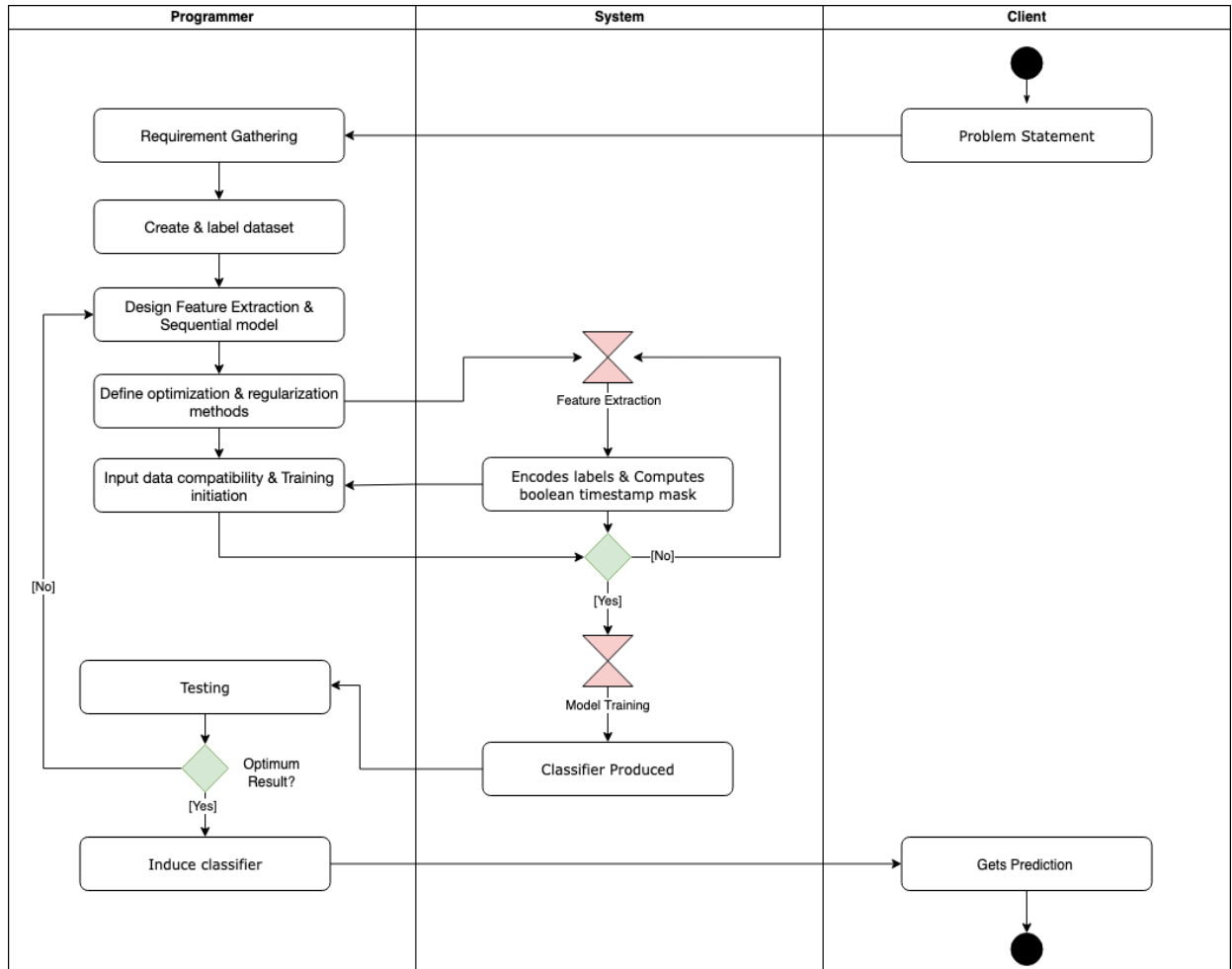


Figure 3.5.1.: Activity diagram for the combined model

CHAPTER 4

RESULTS

This section discusses the results of the proposed models and methodologies to identify and detect instances of rash driving from video footage. We created the project using the Nvidia K80 / T4 (0.82 - 1.59 GHz) GPU of Google Colab [14], with the basic RAM of 12GB and the neural networks were created using the TensorFlow Keras [15] [16] layers. Additionally, the detection module was developed using computer vision methods available in OpenCV [17]. The model performance is first compared with its various architectures and parameter initializations, after which pre-existing architectures are used to draw comparison in terms of performance, accuracy, efficiency, and model complexity.

The RNN classifier was first trained on the dataset without augmentation and parameter tuning. The weights were initialised to the default values available in the respective layers' class constructors. However, we observed that the model was heavily overfitting on the data (figure 4.1). Since the dataset was manually created for this project, there was a possibility that the underperformance of the model could be due to the small and unbalanced dataset.

In order to determine if the problem was with the model or the dataset, the model was trained on a subset of the UCF-101 dataset. The UCF-101 dataset is an action recognition data set of realistic action videos. The action categories can be divided into five types: human-object interaction, body-motion, human-human interaction, playing musical instruments, sports. A subset sampled with five different actions, containing 594 videos for training and 224 videos for testing was used to train the model and test for its performance. The model had overfit on this dataset (figure 4.2.), thus indicating that the parameters of the model had to be tuned.

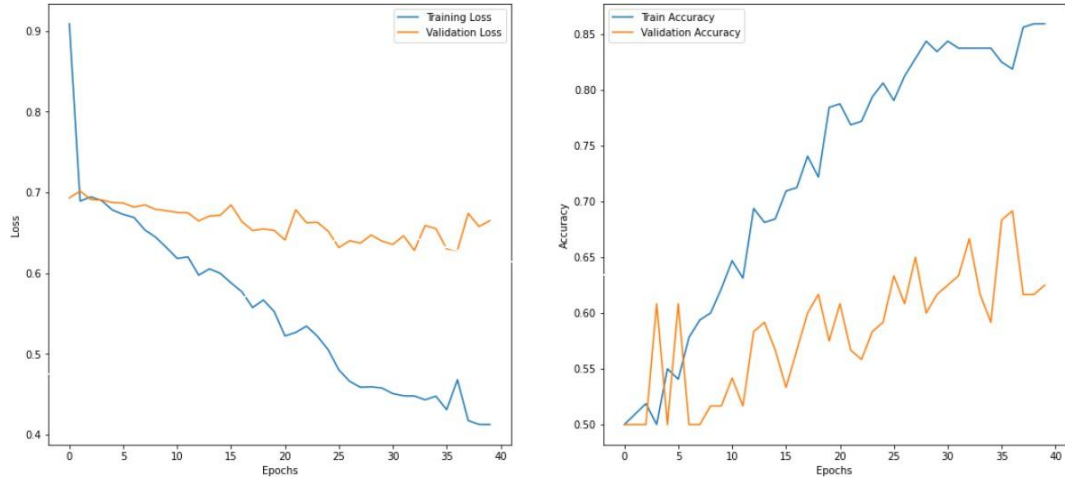


Figure 4.1.: Training and validation accuracies and loss curves before parameter tuning

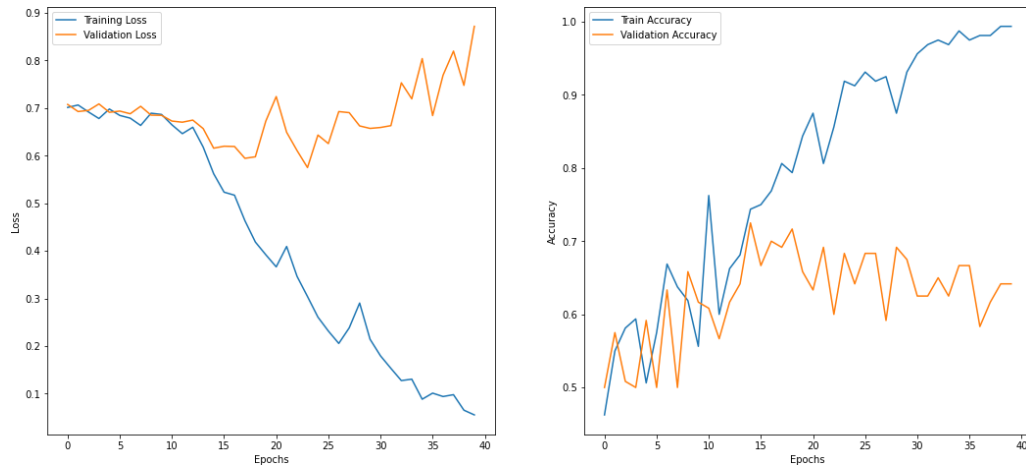


Figure 4.2.: Training and validation accuracies and loss curves of the model trained on the UCF-101 dataset before parameter tuning

To tackle the problem of having a small-scale, unbalanced dataset, we used RandAugment with the parameters specified in the original research [9], to transform and increase the size of the original dataset. Data augmentation helps reduce overfitting during training. We used the Sigmoid activation for the SimpleRNN layers with an L2 regularization (Ridge regression) of

0.01, and the Adam optimizer. As compared to the Lecun, random and GlorotUniform distributions, the GlorotNormal [18] and Orthogonal distributions proved to be the most efficient in minimising the vanishing gradient problem, and hence were used as the kernel and recurrent initializers respectively. The dataset was split into a training and validation set at 20% and was trained for 40 epochs with 10 steps per epoch. Figure 4.3 shows the training and validation accuracy and loss curves plotted against the epochs. The most recent state-of-the-art model architecture proposed for the same purpose is the DriftNet [5], which reached a training and a validation accuracy of 93.75% and 92.5% respectively with 4.69M trainable parameters, has been proved to be more efficient and accurate as compared to other models such as the ConvLSTM, DenseNet and C3D, that were used for the purpose of identifying aggressive driving behaviour. Our model reached a training and a validation accuracy of 90.62% and 90.17% respectively, with 144,026 trainable parameters. Table 4.1 contains information on the various parameters tuned to enhance the performance of the model.

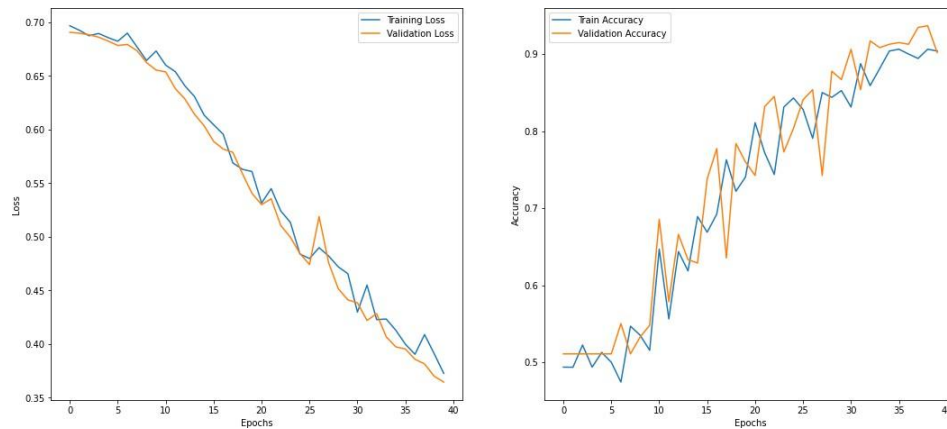


Figure 4.3.: Training and validation accuracies and loss curves after parameter tuning

Additionally, the precision, recall and f1-scores were computed for both the classes using a set of seven videos reserved for testing. (figure 4.5.). The formulas for the three metrics are provided below (figure 4.4.).

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1-score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

Figure 4.4.: Formulas for precision, recall and f1-score

	precision	recall	f1-score	support
Aggressive	0.50	0.25	0.33	4
Non-aggressive	0.40	0.67	0.50	3

Figure 4.5.: Precision, recall and f1-scores for the two classes (aggressive and non-aggressive)

Table 4.1.: Parameters and their values before and after tuning

Parameter	Initial/Default	Modified
Activation	Tanh	Sigmoid
Kernel Initializer	Glorot Uniform	Glorot Normal
Recurrent Initializer	Orthogonal	Orthogonal
Kernel Regularizer	None	L1=0, L2=0.01
Recurrent Regularizer	None	L1=0, L2=0.01
Activity Regularizer	None	L1=0, L2=0.01

Dropout	0.0	0.1
Recurrent Dropout	0.0	0.2
Optimizer	RMSProp	Adam ($\alpha=0.001$)

Varying accuracy and loss curves can be observed with a change in the combination and magnitude of the RandAugment transformations applied before and during feature extraction. Despite having a marginally lower performance as compared to other networks, our architecture is computationally less complex, thereby reducing computation costs and eradicating the need for high-performance computing servers. Additionally, the noisy training can be attributed to the smaller number of epochs, limited batches of training data and a naïve approach to compiling the dataset.

Since the segmentation and detection of the vehicles involved in a rash driving incident is based on frame markers, detecting moving objects and processing of contours using computer vision, the performance of this method to detect the vehicles depends on the model's ability to unfailingly classify video clips based on the presence of aggressive driving patterns.



Figure 4.6.: Annotated Frame Post Detection

CHAPTER 5

CONCLUSION

The aim of the experiment conducted through our project was to prove that understanding and identifying aggressive driving behaviours can be achieved with cost effective methods. To back the success of our project we have been able to achieve substantial training and validation accuracies. We were able to achieve a high training accuracy of 90.62% and are able to achieve 90.17% while validating our model. These numbers were and are achievable with a mere 144,026 trainable parameters. To enable detection of vehicles involved in rash driving, achieve our targets and have high percentage of positive metrics, we have leveraged computer vision tools such as thresholding, frame differencing, processing of contours using areas, and Hausdorff distancing. Through this project, we have succeeded in creating an efficient method that incorporates various modules such as driving pattern classification and vehicle detection by introducing state-of-the-art convolution neural network architectures such as ResNet-50 and the EfficientNet-B7 to enable feature extraction, along with a simple Recurrent Neural Network model, to enable classification. Advancements in deep learning and computer vision techniques have now made it possible to train highly complex deep neural networks for identifying such anomalous behaviours, which would otherwise be computationally costly and require high powered computing servers to be hosted. Currently there are several methods that are used to identify various elements like speeding, reckless driving and various traffic violations, however the solutions currently on hand come at a premium. In terms of hardware costs, software costs, up-keep etc. Our project is working proof of a concept that can help and support the cause and help strive further for safety. Aggressive driving is a menace to all those on and off roads, acts that questions safety and constantly puts lives at risk, and hence deserves solutions like ours that can make the management feasible on a large scale.

CHAPTER 6

FUTURE SCOPE

This project proposes a computationally efficient, cost effective and a well-performing combination of feature extraction, model architecture design and detection algorithm to classify aggressive driving and detect the vehicles involved, from video footage. The experimental results are obtained with the specified regularisation and optimization techniques on the dataset created and compiled by us. Further improvements in the recurrent neural network's framework can be made depending on the availability of large-scale, uniformly distributed and pre-processed datasets along with more RAM size and high-computing servers. Additionally, the feature extraction, image segmentation and Hausdorff distance formulas can be altered with respect to the domain of the problem statement [19] [20]. This project can further be developed to include driver data such as age, gender, stress levels, number of passengers and other elements of traffic such as foreign object intervention, absence of working traffic lights, climatic parameters, and incorrect placement of traffic sign boards. These features can further be included in the classification process and the incidents can be classified into different categories under aggressive or non-aggressive behaviour i.e., can be converted to a multi-class classification problem. The video dataset can also be expanded to include videos shot from dashboard cameras, aerial top views of traffic, drone footage and multi-pivoted CCTV footage. The approach used in this project is essentially for individual small-scale firms to integrate our methodologies with their current video surveillance software and utilise the videos captured by them as a dynamically growing dataset to periodically train the model, thus specializing the algorithms and models according to their use case.

CHAPTER 7

REFERENCES

- [1] K. Gunale and P. Mukherji, “Deep Learning with a Spatiotemporal Descriptor of Appearance and Motion Estimation for Video Anomaly Detection,” *Journal of Imaging*, vol. 4, p. 79, 2018.
- [2] S. Accattoli, P. Sernani, N. Falcionelli, D. Mekuria and A. F. Dragoni, “Violence Detection in Videos by Combining 3D Convolutional Neural Networks and Support Vector Machines,” *Applied Artificial Intelligence*, vol. 34, pp. 1-16, 2020.
- [3] Z. Fu, W. Hu and T. Tan, “Similarity based vehicle trajectory clustering and anomaly detection,” in *IEEE International Conference on Image Processing 2005*, 2005.
- [4] D. Shinar and R. Compton, “Aggressive Driving: An Observational Study of Driver, Vehicle, and Situational Variables,” *Accident; analysis and prevention*, vol. 36, pp. 429-437, 2004.
- [5] A. Noor, B. Benjdira, A. Ammar and A. Koubaa, “DriftNet: Aggressive Driving Behaviour Detection using 3D Convolutional Neural Networks,” in *First International Conference of Smart Systems and Emerging Technologies (SMARTTECH)*, Riyadh, 2020.
- [6] H. Alkinani, W. Khan and Q. Arshad, “Detecting Human Driver Inattentive and Aggressive Driving Behavior Using Deep Learning: Recent Advances, Requirements and Open Challenges,” *IEEE Access*, vol. PP, pp. 1-1, 2020.

- [7] K. He, X. Zhang, S. Ren and J. Sun, “Deep Residual Learning for Image Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition*, 2016.
- [8] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *International Conference on Machine Learning*, 2019.
- [9] E. D. Cubuk, B. Zoph, J. Shlens and Q. V. Le, “RandAugment: Practical automated data augmentation with a reduced search space,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2019.
- [10] S. Lamba and N. Nain, “A Literature Review on Crowd Scene Analysis and Monitoring,” *International Journal of Urban Design for Ubiquitous Computing*, vol. 4, pp. 9-20, 2016.
- [11] W. Ge, R. Collins and B. Ruback, “Automatically detecting the small group structure of a crowd,” in *Applications of Computer Vision (WACV)*, 2010.
- [12] X. Wang, K. Tieu and E. Grimson, “Learning Semantic Scene Models by Trajectory Analysis,” in *Proceedings of the 9th European conference on Computer Vision - Volume Part III*, 2006.
- [13] A. A. Taha and A. Hanbury, “An efficient algorithm for calculating the exact Hausdorff distance,” *IEEE transactions on pattern analysis and machine intelligence*, pp. 1-1, 2015.
- [14] E. Bisong, “Google Colaboratory,” in *Building Machine Learning and Deep Learning Models on Google Cloud Platform: A Comprehensive Guide for Beginners*, Apress, 2019, pp. 59-64.
- [15] F. Chollet and others, “Keras,” GitHub, 2015.

- [16] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard and others, *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*, 2015.
- [17] G. Bradski, "The OpenCV Library," *Dr. Dobbs's Journal of Software Tools*, 2000.
- [18] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," *Journal of Machine Learning Research - Proceedings Track*, vol. 9, pp. 249-256, 2010.
- [19] J. Ngiam, D. Peng, V. Vasudevan, S. Kornblith, Q. V. Le and R. Pang, *Domain Adaptive Transfer Learning with Specialist Models*, 2018.
- [20] L. Zhu, S. Arik, Y. Yang and T. Pfister, *Learning to Transfer Learn: Reinforcement Learning-Based Selection for Adaptive Transfer Learning*, 2019.
- [21] Y. Li, H. Liu, X. Zheng, Y. Han and L. Li, "A Top-Bottom Clustering Algorithm Based on Crowd Trajectories for Small Group Classification," *IEEE Access*, vol. PP, pp. 1-1, 2019.
- [22] G. Huang, Y. Sun, Z. Liu, D. Sedra and K. Weinberger, "Deep Networks with Stochastic Depth," in *European Conference on Computer Vision*, 2016.
- [23] D. Masters, A. Labatie, Z. Eaton-Rosen and C. Luschi, *Making EfficientNet More Efficient: Exploring Batch-Independent Normalization, Group Convolutions and Reduced Resolution Training*, 2021.
- [24] I. Bello, W. Fedus, X. Du, E. D. Cubuk, A. Srinivas, T.-Y. Lin, J. Shlens and B. Zoph, *Revisiting ResNets: Improved Training and Scaling Strategies*, 2021.

APPENDIX 1

This section contains details on the language, software and packages used in our project.

This project is developed in Python, which is a general-purpose interpreted, interactive, object oriented and high-level programming language. It offers concise and readable code. Despite being highly complex with versatile workflows, the AI and ML algorithms, when written in Python, can help the developers create robust and reliable machine intelligent systems. The list of Python packages used in our project are:

- **Numpy:** It is a Python library that provides a multidimensional array object, various derived objects (such as masked arrays and matrices), and an assortment of routines for fast operations on arrays, including mathematical, logical, shape manipulation, sorting, selecting, I/O, discrete Fourier transforms, basic linear algebra, basic statistical operations, random simulation, etc. It is the fundamental package for scientific computing in Python.
- **Pandas:** It is a Python package that provides fast, flexible, and expressive data structures designed to make working with "relational" or "labeled" data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, real world data analysis in Python. Additionally, it has the broader goal of becoming the most powerful and flexible open source data analysis / manipulation tool available in any language.
- **Imgaug:** It is a library for image augmentation in machine learning experiments. It supports a wide range of augmentation techniques, allows to easily combine these and to execute them in random order or on multiple CPU cores, has a simple yet powerful stochastic interface and can not only augment images, but also keypoints/landmarks, bounding boxes, heatmaps and segmentation maps.
- **OpenCV-python:** OpenCV (Open Source Computer Vision Library) is an open source computer vision and machine learning software library. OpenCV was built to provide a

common infrastructure for computer vision applications and to accelerate the use of machine perception in the commercial products. The library has more than 2500 optimized algorithms, which includes a comprehensive set of both classic and state-of-the-art computer vision and machine learning algorithms. These algorithms can be used to detect and recognize faces, identify objects, classify human actions in videos, track camera movements, track moving objects, extract 3D models of objects, produce 3D point clouds from stereo cameras, stitch images together to produce a high resolution image of an entire scene, find similar images from an image database, remove red eyes from images taken using flash, follow eye movements, recognize scenery and establish markers to overlay it with augmented reality, etc.

- **PIL:** The Python Imaging Library was developed by Fredrik Lundh and Contributors. It adds image processing capabilities to the Python interpreter and is designed for fast access to data stored in a few basic pixel formats.
- **Matplotlib:** It is a comprehensive library for creating static, animated, and interactive visualizations in Python. It can create publication quality plots, make interactive figures that can zoom, pan, update, customize visual style and layout, export to many file formats and can be embedded in JupyterLab and Graphical User Interfaces.
- **Keras:** It is an open-source software library that provides a Python interface for artificial neural networks. Keras acts as an interface for the TensorFlow library. Up until version 2.3, Keras supported multiple backends, including TensorFlow, Microsoft Cognitive Toolkit, Theano, and PlaidML.
- **Tensorflow:** It is a free and open-source software library for machine learning and artificial intelligence. It can be used across a range of tasks but has a particular focus on training and inference of deep neural networks. TensorFlow was developed by the Google Brain team for internal Google use in research and production. TensorFlow can be used in a wide variety of programming languages, most notably Python, as well as Javascript, C++, and Java. It has a comprehensive, flexible ecosystem of tools, libraries and community resources that lets researchers push the state-of-the-art in ML and

developers easily build and deploy ML powered applications.

- **Scikit-learn:** It is a free software machine learning library for the Python programming language. It features various classification, regression and clustering algorithms including support-vector machines, random forests, gradient boosting, k-means and DBSCAN, and is designed to interoperate with the Python numerical and scientific libraries NumPy and SciPy.
- **SciPy:** It is a free and open-source Python library used for scientific computing and technical computing. It contains modules for optimization, linear algebra, integration, interpolation, special functions, signal and image processing, etc. The basic data structure used by SciPy is a multidimensional array provided by the NumPy module.
- **Tkinter:** It is the standard GUI library for Python. Python when combined with Tkinter provides a fast and easy way to create GUI applications. Tkinter provides a powerful object-oriented interface to the Tk GUI toolkit. Tkinter calls are translated into Tcl commands, which are fed to this embedded interpreter, thus making it possible to mix Python and Tcl in a single application.

APPENDIX 2

This section contains the code for training the model and developing the object detection algorithm used in this project, and the GUI for the output window.

Model training and vehicle detection:

```
image_size=(260,260)
path1= r"/content/drive/MyDrive/major project/videos/"
```

```
df_train=pd.read_csv(r"/content/drive/MyDrive/major project/train.csv")
df_test=pd.read_csv(r"/content/drive/MyDrive/major project/test.csv")
```

```
df_train.head()
```

	path	label
0	/content/drive/MyDrive/major project/videos/tr...	aggressive
1	/content/drive/MyDrive/major project/videos/tr...	aggressive
2	/content/drive/MyDrive/major project/videos/tr...	aggressive
3	/content/drive/MyDrive/major project/videos/tr...	aggressive
4	/content/drive/MyDrive/major project/videos/tr...	aggressive

```
df_test.head()
```

	path	label
0	/content/drive/MyDrive/major project/videos/te...	aggressive
1	/content/drive/MyDrive/major project/videos/te...	aggressive
2	/content/drive/MyDrive/major project/videos/te...	aggressive
3	/content/drive/MyDrive/major project/videos/te...	aggressive
4	/content/drive/MyDrive/major project/videos/te...	aggressive

```

for pdf in [p1_train, p1_test]:
    fps=[]
    for pi in pdf:
        cap=cv2.VideoCapture(pi)
        ret=True
        cfr=0
        while(ret):
            ret, frame=cap.read()
            if(ret==False):
                break
            cfr+=1
            if(cv2.waitKey(15) & 0xFF==ord('q')):
                break
        cap.release()
        cv2.destroyAllWindows()
        fps.append(cfr)
    print(max(fps))
    if(MAX_SEQ_LENGTH<max(fps)):
        MAX_SEQ_LENGTH=max(fps)
    print(fps)
MAX_SEQ_LENGTH+=1
print(MAX_SEQ_LENGTH)

```

```

def load_video(paths, path):
    cap = cv2.VideoCapture(path)
    frames = []
    while True:
        ret, frame = cap.read()
        if ret==False:
            break
        frames.append(cv2.resize(frame, IMG_SIZE))
    cap.release()
    ind=paths.index(path)
    if(len(paths)==len(p1_train)):
        max_lim=p1_train_frames[ind]
    else:
        max_lim=p1_test_frames[ind]
    return np.array(frames[:ind])

```

```

def loadvideo(path):
    cap = cv2.VideoCapture(path)
    frames = []
    while True:
        ret, frame = cap.read()
        if ret==False:
            break
        frames.append(cv2.resize(frame, IMG_SIZE))
    cap.release()
    return np.array(frames)

```

```

label_processor = keras.layers.StringLookup(
    num_oov_indices=0, vocabulary=np.unique(df_train["label"])
)
labels=label_processor.get_vocabulary()
print(labels)

```

```
['aggressive', 'non-aggressive']
```

```

### RandAugment

rand_aug = imgaug.augmenters.RandAugment(n=2, m=9)
def augment(images):
    #image = tf.cast(image, tf.uint8)
    return rand_aug(images=images)

```

```

def get_resnet_shape(n):
    a=b=int(n**0.5)
    while(True):
        if(a*b==n or a==n):
            break
        else:
            a+=1
            if(a*b>n):
                b-=1
    return ((a, b)+(3,))

def combined_cnn(input_shape, num_classes):
    ennb7 = nn.EfficientNetB7(weights='imagenet', include_top=False,
                               input_shape=input_shape, classes=2)

    inputs = keras.Input(shape=input_shape)
    #x = data_augmentation(inputs)
    l = tf.keras.applications.efficientnet.preprocess_input(inputs)
    l = ennb7(l)
    shp=ennb7.output_shape[1:]
    n=(shp[0]*shp[1]*shp[2])/3
    reshaped_shape=get_resnet_shape(n)
    l = tf.keras.layers.Reshape(target_shape=reshaped_shape)(l)

    resnet50=nn.ResNet50(include_top=False, weights = 'imagenet',
                          input_shape=reshaped_shape, classes=2, pooling='avg')
    l = tf.keras.applications.resnet50.preprocess_input(l)
    l = resnet50(l)
    return keras.Model(inputs, l)

model1 = combined_cnn(input_shape=image_size + (3,), num_classes=2)
tf.keras.utils.plot_model(model1, show_shapes=True)

```

Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb7_notop.h5
 258080768/258076736 [=====] - 3s 0us/step
 2580808960/258076736 [=====] - 3s 0us/step
 Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_kernels_notop.h5
 94773248/94765736 [=====] - 1s 0us/step
 94781440/94765736 [=====] - 1s 0us/step

input_3	input:	[(None, 260, 260, 3)]	[(None, 260, 260, 3)]
InputLayer	output:		

efficientnetb7	input:	(None, 260, 260, 3)	(None, 9, 9, 2560)
Functional	output:		

reshape	input:	(None, 9, 9, 2560)	(None, 270, 256, 3)
Reshape	output:		

tf.__operators__.getitem	input:	(None, 270, 256, 3)	(None, 270, 256, 3)
SlicingOpLambda	output:		

tf.nn.bias_add	input:	(None, 270, 256, 3)	(None, 270, 256, 3)
TFOpLambda	output:		

resnet50	input:	(None, 270, 256, 3)	(None, 2048)
Functional	output:		

```

def feature_extraction(df, paths, fname):
    num_samples = len(df)
    pad_bool = []
    feats = []
    lbs = df['label'].values
    lbs = label_processor(lbs[... , None]).numpy()
    with open(fname, 'w') as f:
        for idx, path in enumerate(paths):
            frames = load_video(paths, path)
            frames = augment(frames)
            length=frames.shape[0]
            temp_frame_mask = [0]*MAX_SEQ_LENGTH
            temp_frame_features = [0]*MAX_SEQ_LENGTH
            for j in range(length):
                temp_frame_features[j] = model1.predict(augment(frames[None, j, :]))
            temp_frame_mask[:length] = [1]*length
            feats.append(np.array(temp_frame_features).squeeze())
            #np.savetxt(f, np.array(temp_frame_features).squeeze().reshape(-1), delimiter=', ', fmt='%s')
            pad_bool.append(np.array(temp_frame_mask, dtype="bool").squeeze())
    f.close()
    return feats, pad_bool, lbs

inp_mask, lb = feature_extraction(df_train, r"/content/train_features.txt")
test_mask, lb_test = feature_extraction(df_test, r"/content/test_features.txt")
  
```

```

inp_set=np.array(inp_set)
test_set=np.array(test_set)
print(inp_set.shape, test_set.shape)

for i in range(inp_set.shape[0]):
    for j in range(inp_set.shape[1]):
        if(type(inp_set[i][j]) is int):
            inp_set[i][j]= np.zeros((2048,))
        else:
            inp_set[i][j]=inp_set[i][j].squeeze()

for i in range(test_set.shape[0]):
    for j in range(test_set.shape[1]):
        if(type(test_set[i][j]) is int):
            test_set[i][j]= np.zeros((2048,))
        else:
            test_set[i][j]=test_set[i][j].squeeze()
train_set=[]
for i in range(inp_set.shape[0]):
    train_set.append(np.stack(inp_set[i]))
train_set=np.array(train_set)
print(train_set.shape)
testset=[]
for i in range(test_set.shape[0]):
    testset.append(np.stack(test_set[i]))
#testset.append(test_last)
testset=np.array(testset)
print(testset.shape)
inp_mask=np.array(inp_mask)
test_mask=np.array(test_mask)

lbs_train=np.array(list(map(float, lb.squeeze()))))
lbs_test=np.array(list(map(float, lb_test.squeeze()))))
lbs_train1=[]
for i in lbs_train:
    if(i==0):
        lbs_train1.append(np.array([1.0, 0.0]))
    else:
        lbs_train1.append(np.array([0.0, 1.0]))
lbs_train1=np.array(lbs_train1)
print(lbs_train1.shape)

lbs_test1=[]
for i in lbs_test:
    if(i==0):
        lbs_test1.append(np.array([1.0, 0.0]))
    else:
        lbs_test1.append(np.array([0.0, 1.0]))
lbs_test1=np.array(lbs_test1)
print(lbs_test1.shape)

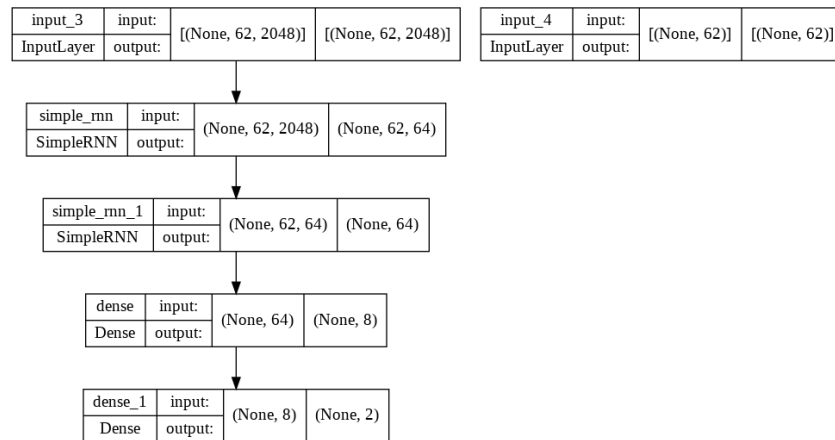
```



```
def get_sequence_model():
    feats = keras.Input((MAX_SEQ_LENGTH, 2048))
    msk = keras.Input((MAX_SEQ_LENGTH, dtype="bool"))
    l = SimpleRNN(64, activation='sigmoid', kernel_initializer='glorot_normal',
                  recurrent_initializer='orthogonal', kernel_regularizer=L1L2(l1=0.0, l2=0.01),
                  recurrent_regularizer=L1L2(l1=0.0, l2=0.01), activity_regularizer=L1L2(l1=0.0, l2=0.01),
                  kernel_constraint=None, recurrent_constraint=None, dropout=0.1, recurrent_dropout=0.2,
                  return_sequences=True, unroll=False)(feats, mask=msk)
    l = SimpleRNN(64, activation='sigmoid', kernel_initializer='glorot_normal',
                  recurrent_initializer='orthogonal', kernel_regularizer=L1L2(l1=0.0, l2=0.01),
                  recurrent_regularizer=L1L2(l1=0.0, l2=0.01), activity_regularizer=L1L2(l1=0.0, l2=0.01),
                  kernel_constraint=None, recurrent_constraint=None, dropout=0.1, recurrent_dropout=0.2)(l)
    l = Dense(8, activation="sigmoid")(l)
    output = Dense(2, activation="softmax")(l)
    rnn_model = keras.Model([feats, msk], output)
    rnn_model.compile(loss=tf.keras.losses.BinaryCrossentropy(from_logits=False), optimizer=Adam(lr=0.001),
                      metrics=["accuracy"])

    return rnn_model

seq_model = get_sequence_model()
tf.keras.utils.plot_model(seq_model, show_shapes=True)
```



```
# Utility for running experiments.
def run_experiment():
    filepath = "weights"
    checkpoint = keras.callbacks.ModelCheckpoint(filepath, monitor="val_accuracy", mode='max',
                                                  save_weights_only=True, save_best_only=True, verbose=1)
    #earlystop = keras.callbacks.EarlyStopping(monitor="accuracy", patience=5, verbose=1, mode="auto")
    seq_model = get_sequence_model()
    history = seq_model.fit([train_set, inp_mask], lbs_train1, verbose=1, validation_data=([testset, test_mask], lbs_test1),
                           shuffle=True, epochs=40, steps_per_epoch=10, callbacks=[checkpoint])

    return history, seq_model

hist, sequence_model = run_experiment()
#sequence_model.save("final_model.h5")
```

```

Epoch 31/40
10/10 [=====] - ETA: 0s - loss: 0.4298 - accuracy: 0.8313
Epoch 31: val_accuracy improved from 0.87773 to 0.90611, saving model to weights
10/10 [=====] - 3s 271ms/step - loss: 0.4298 - accuracy: 0.8313 - val_loss: 0.4384 - val_accuracy: 0.9061
Epoch 32/40
10/10 [=====] - ETA: 0s - loss: 0.4550 - accuracy: 0.8875
Epoch 32: val_accuracy did not improve from 0.90611
10/10 [=====] - 3s 268ms/step - loss: 0.4550 - accuracy: 0.8875 - val_loss: 0.4220 - val_accuracy: 0.8537
Epoch 33/40
10/10 [=====] - ETA: 0s - loss: 0.4228 - accuracy: 0.8590
Epoch 33: val_accuracy improved from 0.90611 to 0.91703, saving model to weights
10/10 [=====] - 3s 270ms/step - loss: 0.4228 - accuracy: 0.8590 - val_loss: 0.4283 - val_accuracy: 0.9170
Epoch 34/40
10/10 [=====] - ETA: 0s - loss: 0.4233 - accuracy: 0.8813
Epoch 34: val_accuracy did not improve from 0.91703
10/10 [=====] - 3s 268ms/step - loss: 0.4233 - accuracy: 0.8813 - val_loss: 0.4067 - val_accuracy: 0.9083
Epoch 35/40
10/10 [=====] - ETA: 0s - loss: 0.4129 - accuracy: 0.9038
Epoch 35: val_accuracy did not improve from 0.91703
10/10 [=====] - 3s 274ms/step - loss: 0.4129 - accuracy: 0.9038 - val_loss: 0.3974 - val_accuracy: 0.9127
Epoch 36/40
10/10 [=====] - ETA: 0s - loss: 0.3999 - accuracy: 0.9062
Epoch 36: val_accuracy did not improve from 0.91703
10/10 [=====] - 3s 275ms/step - loss: 0.3999 - accuracy: 0.9062 - val_loss: 0.3954 - val_accuracy: 0.9148
Epoch 37/40
10/10 [=====] - ETA: 0s - loss: 0.3904 - accuracy: 0.9000
Epoch 37: val_accuracy did not improve from 0.91703
10/10 [=====] - 3s 276ms/step - loss: 0.3904 - accuracy: 0.9000 - val_loss: 0.3858 - val_accuracy: 0.9127
Epoch 38/40
10/10 [=====] - ETA: 0s - loss: 0.4088 - accuracy: 0.8942
Epoch 38: val_accuracy improved from 0.91703 to 0.93450, saving model to weights
10/10 [=====] - 3s 277ms/step - loss: 0.4088 - accuracy: 0.8942 - val_loss: 0.3815 - val_accuracy: 0.9345
Epoch 39/40
10/10 [=====] - ETA: 0s - loss: 0.3911 - accuracy: 0.9062
Epoch 39: val_accuracy improved from 0.93450 to 0.93668, saving model to weights
10/10 [=====] - 3s 275ms/step - loss: 0.3911 - accuracy: 0.9062 - val_loss: 0.3700 - val_accuracy: 0.9367
Epoch 40/40
10/10 [=====] - ETA: 0s - loss: 0.3727 - accuracy: 0.9038
Epoch 40: val_accuracy did not improve from 0.93668
10/10 [=====] - 3s 270ms/step - loss: 0.3727 - accuracy: 0.9038 - val_loss: 0.3645 - val_accuracy: 0.9017

```

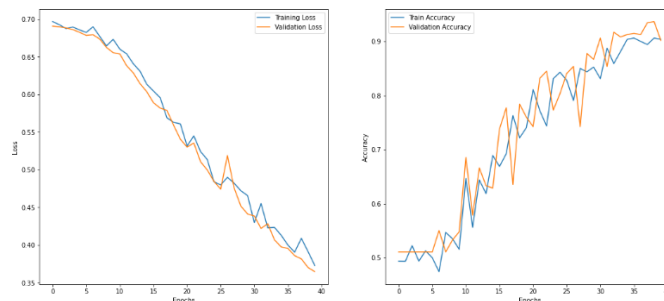
```

plt.figure(figsize=(18,8))

plt.subplot(1,2,1)
plt.plot(hist.history['loss'], label='Training Loss')
plt.plot(hist.history['val_loss'], label='Validation Loss')
plt.legend()
plt.xlabel('Epochs')
plt.ylabel('Loss')

plt.subplot(1,2,2)
plt.plot(hist.history['accuracy'], label='Train Accuracy')
plt.plot(hist.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.show()

```



```
target = ["Aggressive","Non-aggressive"]
print(classification_report(lbs_test, np.argmax(seq_model.predict([testset, test_mask]),axis=1), target_names=target))
```

	precision	recall	f1-score	support
Aggressive	0.50	0.25	0.33	4
Non-aggressive	0.40	0.67	0.50	3
accuracy			0.43	7
macro avg	0.45	0.46	0.42	7
weighted avg	0.46	0.43	0.40	7

```
def isRectangleOverlap(R1, R2):
    if (R1[0]>=R2[2]) or (R1[2]<=R2[0]) or (R1[3]<=R2[1]) or (R1[1]>=R2[3]):
        return False
    else:
        dx = min(R1[2], R2[2]) - max(R1[0], R2[0])
        dy = min(R1[3], R2[3]) - max(R1[1], R2[1])
        if (dx>=0) and (dy>=0):
            area=dx*dy
        else:
            area=0
        arx=(R1[2]-R1[0])*(R1[3]-R1[1])
        ary=(R2[2]-R2[0])*(R2[3]-R2[1])
        if(area>0 and (arx-area)>0 and (ary-area)>0):
            return True
        else:
            return False
```

```
def get_annotated_frames(frames, min_area):
    kernel = np.ones((3,3),np.uint8)
    end_frames=[]
    max_cntrs=[]
    #global hdf5
    for i in range(len(frames)-1):
        grayA = cv2.cvtColor(frames[i], cv2.COLOR_BGR2GRAY)
        grayB = cv2.cvtColor(frames[i+1], cv2.COLOR_BGR2GRAY)
        diff_image = cv2.absdiff(grayB, grayA)
        ret, thresh = cv2.threshold(diff_image, 30, 255, cv2.THRESH_BINARY)
        dilated = cv2.dilate(thresh,kernel,iterations = 1)
        contours, hierarchy = cv2.findContours(dilated.copy(), cv2.RETR_TREE,cv2.CHAIN_APPROX_NONE)
        valid_cntrs = []
        for cntr in contours:
            if (cv2.contourArea(cntr) >= 30):
                valid_cntrs.append(cntr)
        vcs={}
        vcs1=[]
        for ci in range(1,len(valid_cntrs)):
            x1,y1,w1,h1 = cv2.boundingRect(valid_cntrs[ci-1])
            x2,y2,w2,h2 = cv2.boundingRect(valid_cntrs[ci])
            if(isRectangleOverlap([x1,y1,x1+w1,y1+h1],[x2,y2,x2+w2,y2+h2])):
                mask1=cv2.contourArea(valid_cntrs[ci-1])
                mask2=cv2.contourArea(valid_cntrs[ci])
                inp_arr1=valid_cntrs[ci-1]
                inp_arr2=valid_cntrs[ci]
                if(valid_cntrs[ci-1].shape!=valid_cntrs[ci].shape):
                    if(inp_arr1.shape[0]<inp_arr2.shape[0]):
                        np.append(inp_arr1,np.zeros((inp_arr2.shape[0]-inp_arr1.shape[0],1,2)),axis=0)
                    else:
                        np.append(inp_arr2,np.zeros((inp_arr1.shape[0]-inp_arr2.shape[0],1,2)),axis=0)
```

```

haus_dist=hausdorff_distance(inp_arr1,inp_arr2)
if(haus_dist<1):
    haus_dist=1
#hdfs=[(ci, haus_dist)]+hdfs
mask_cmb=cv2.bitwise_and(mask1, mask2)
#print(haus_dist, sum(mask_cmb.squeeze()))
if(sum(mask_cmb.squeeze())>min_area and min_area<haus_dist*IMG_SIZE[0]):
    #print(sum(mask_cmb.squeeze()))
    vcs[sum(mask_cmb.squeeze())]=(valid_cntrs[ci-1], valid_cntrs[ci])
if(len(vcs)>0):
    vcs1=[]
    for ki in sorted(vcs.keys(), reverse=True):
        vcs1.append(vcs[ki][0])
        vcs1.append(vcs[ki][1])
    dny = frames[i].copy()
    #cv2.drawContours(dny, vcs1[:2], -1, (0,0,255), 2)
    start_point=(min(cv2.boundingRect(vcs1[0])[0], cv2.boundingRect(vcs1[1])[0]),
        max(cv2.boundingRect(vcs1[0])[1]+cv2.boundingRect(vcs1[0])[3], cv2.boundingRect(vcs1[1])[1]+cv2.boundingRect(vcs1[1])[3]))
    end_point=(max(cv2.boundingRect(vcs1[0])[0]+cv2.boundingRect(vcs1[0])[2], cv2.boundingRect(vcs1[1])[0]+cv2.boundingRect(vcs1[1])[2]),
        min(cv2.boundingRect(vcs1[0])[1], cv2.boundingRect(vcs1[1])[1]))
    #print(start_point, end_point)
    dny=cv2.rectangle(dny, start_point, end_point, (0,0,200), 2)
    end_frames.append(dny)
    max_cntrs.append((start_point[1]-end_point[1])*(end_point[0]-start_point[0]))
return(end_frames, max_cntrs)

end_frames, max_cntrs= get_annotated_frames(frames, 250)

```

Output module:

```

from tkinter import *
import matplotlib
from tkinter import messagebox, filedialog
from PIL import ImageTk, Image
from tkvideo import tkvideo
from detection import get_annotated_frames
from classifier import clsres
import cv2
import os

```

```

vid_path=""
number=2

```

```

col1=(209, 212, 218)
col2=(226,226,226)
transparent=(225,225,224)
pop=(11, 16, 46)

```

```

def rgb_col(rgba):

```

```

return "#%02x%02x%02x" % rgba
#return matplotlib.colors.to_hex(rgba)

def go_back():
    for wi in fr1.winfo_children():
        wi.place_forget()
    fr1.place_forget()

def get_main_frame():
    but1.place(relx=0.37, rely=0.06,width=100,height=30)
    but_back.place(relx=0.04, rely=0.9,width=24,height=24)
    fr1.place(relx=0.499, rely=0.185)
    fr1.pack_propagate(0)

def upload_file():
    global vid_path
    vid_path=filedialog.askopenfilename()
    video_label = Label(fr1)
    video_label.place(relx=0.18, rely=0.18)
    player = tkvideo(vid_path, video_label,loop = 1, size = (250, 250))
    player.play()
    probs = clsres(vid_path)
    if(probs!=0):
        __ = str(float(probs)*100)
        res_text = ' Aggressiveness: '+__+'%'
        but1.place(relx=0.11, rely=0.06,width=100,height=30)
        but_exfr.place(relx=0.6, rely=0.06,width=120,height=30)
    else:
        res_text = 'No signs of aggressive driving'
        if(but_exfr.winfo_ismapped()):
            but_exfr.place_forget()
        but1.place(relx=0.37, rely=0.06,width=100,height=30)
    lb1=Label(fr1, text=res_text,font=('Comic Sans MS',11), bg=rgb_col(col2))
    lb1.place(relx=0.12,rely=0.82,width=300,height=40)
    lb1.propagate(0)

def display_img():
    n=250
    i=0
    j=0
    end_frames, max_cntrs= get_annotated_frames(vid_path, 250, 40)
    path=r"final frames/"
    for f in os.listdir(path):
        os.remove(os.path.join(path, f))
    for ind, im in enumerate(end_frames):

```

```

        cv2.imwrite(path+'frame'+str(ind)+'.png', im)
    r1=Toplevel(root)
    r1.configure(bg=rgb_col(pop))
    r1.geometry('370x430')
    lbhead=Label(r1, text='EXTRACTED FRAMES', font=('Comic Sans MS',12),
bg=rgb_col(col2), foreground=rgb_col(pop))
    lbhead.place(relx=0.05, rely=0.105, width=320, height=40)
    canh= len(end_frames)*275+50
    t= Canvas(r1, bg=rgb_col(col2), scrollregion=(0,0,0,canh))
    t.place(relx=0.05, rely=0.21, width=320, height=320)
    y_scroll= Scrollbar(r1, orient='vertical', command=t.yview)
    y_scroll.place(relx=1, rely=0, relheight=1, anchor='ne')
    t.configure(yscrollcommand=y_scroll.set)
    t.configure(scrollregion=t.bbox('all'))
    i=160
    j=160
    for name in os.listdir(path):
        print(name)
        image = Image.open(path + name).resize((n,n))
        photo = ImageTk.PhotoImage(image)
        lbimg=Label(t, image=photo)
        lbimg.image=photo
        t.create_window(i,j,window=lbimg)
        j+=275

root = Tk()
root.geometry('966x644+0+0')
root.wm_attributes('-transparentcolor', rgb_col(transparent))

bgimg=ImageTk.PhotoImage(Image.open("bg1_new1.png").resize((966,644)))
lb_root=Label(root, image=bgimg)
lb_root.place(relheight=1, relwidth=1, relx=0, rely=0)
lb_root.image=bgimg
lb_root.pack_propagate(0)

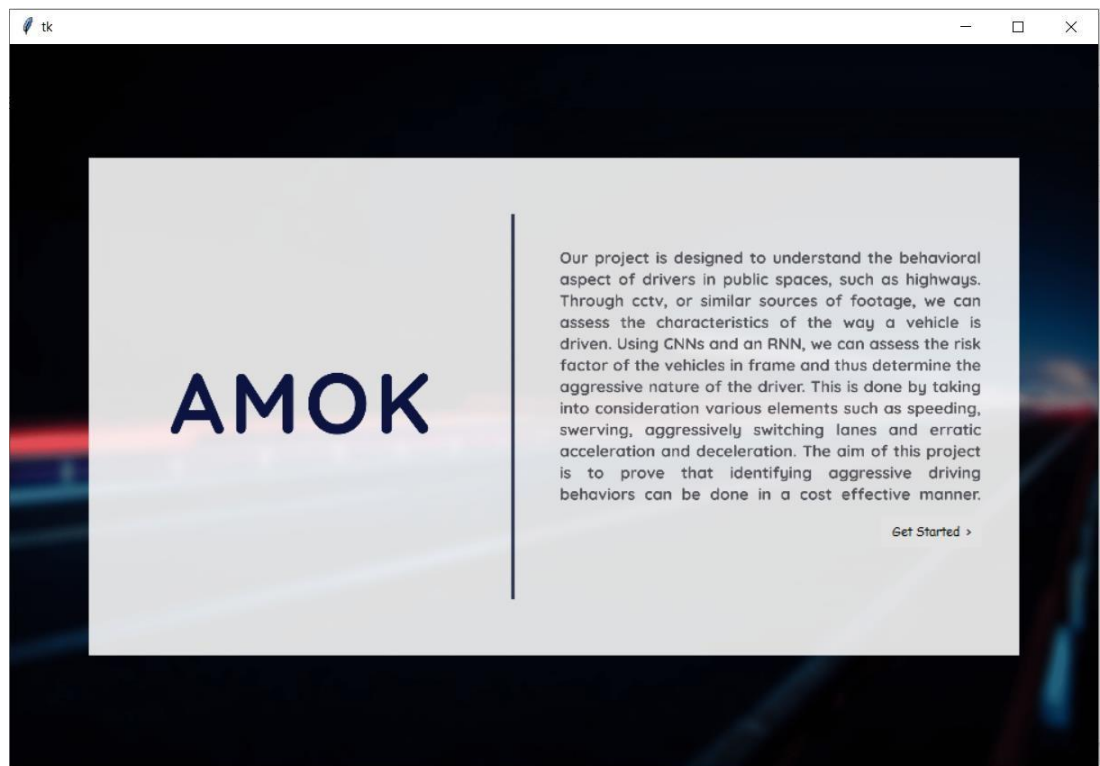
butst=Button(root, text='Get Started >', command=get_main_frame, font=('Comic Sans
MS',8), relief='flat', bg=rgb_col(col2))
butst.place(relx=0.8, rely=0.65, width=90, height=28)

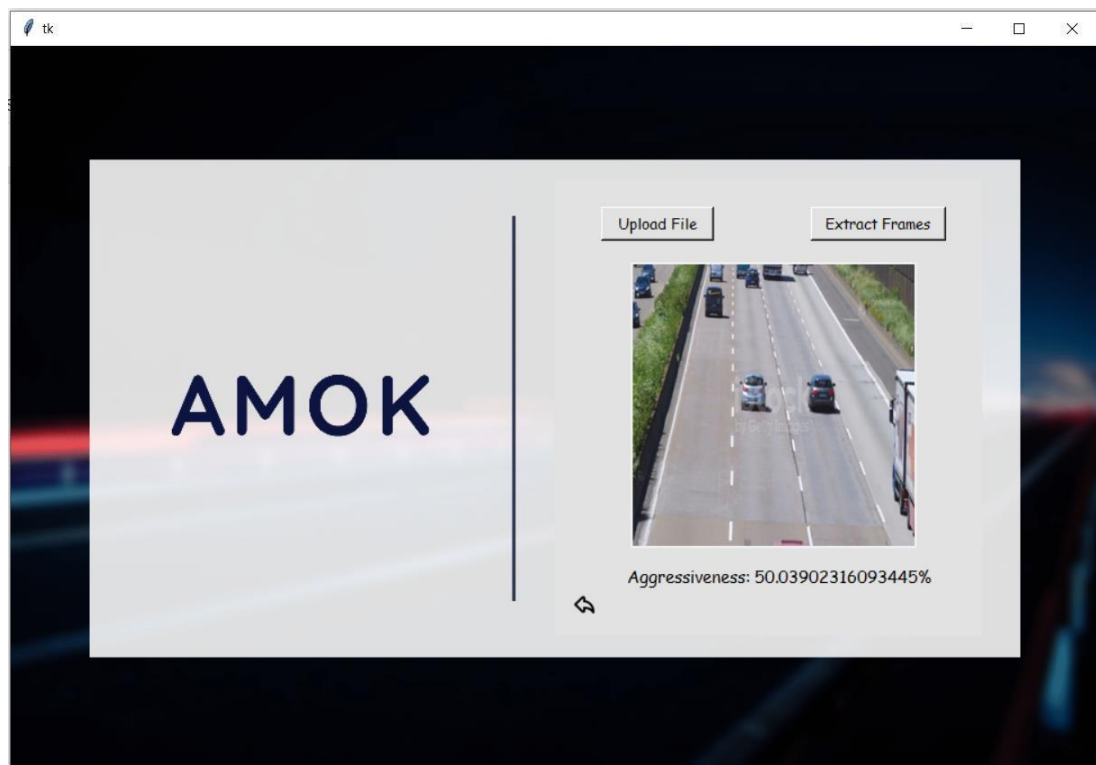
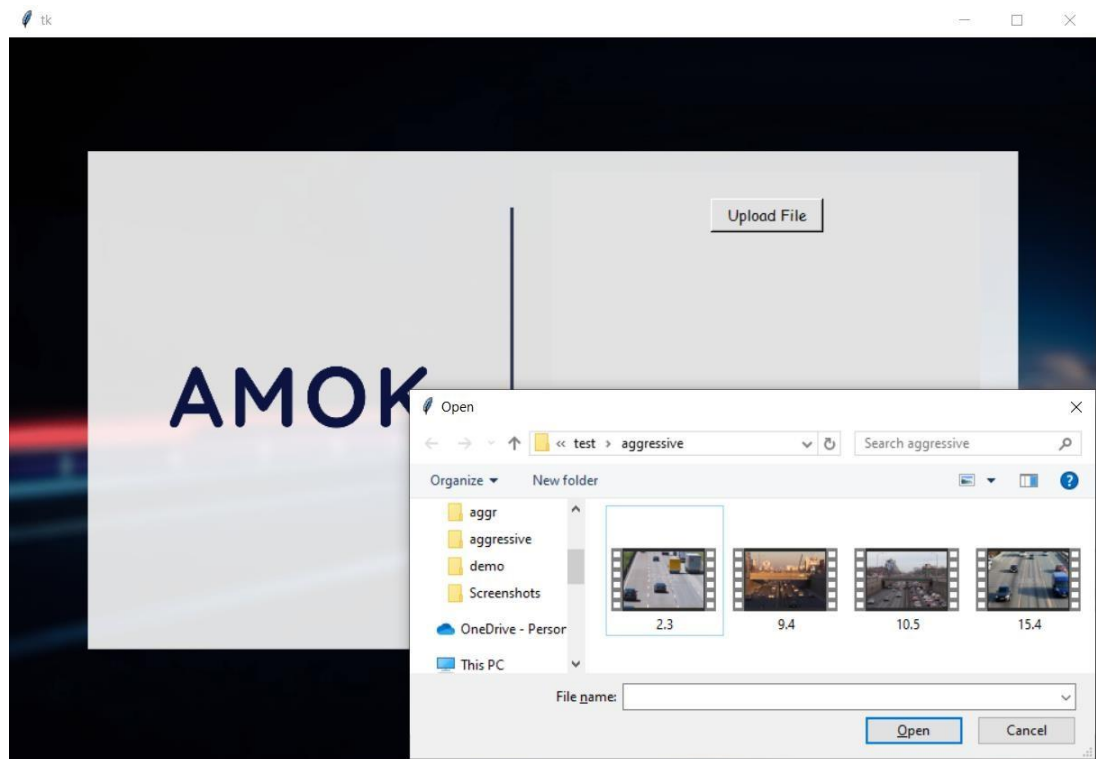
fr1=Frame(root, height=405, width=380, bg=rgb_col(col2))
but1=Button(fr1, text='Upload File', command=upload_file, font=('Comic Sans
MS',10), bg=rgb_col(col2))
but_exfr=Button(fr1, text='Extract Frames', command=display_img, font=('Comic Sans
MS',10), bg=rgb_col(col2))

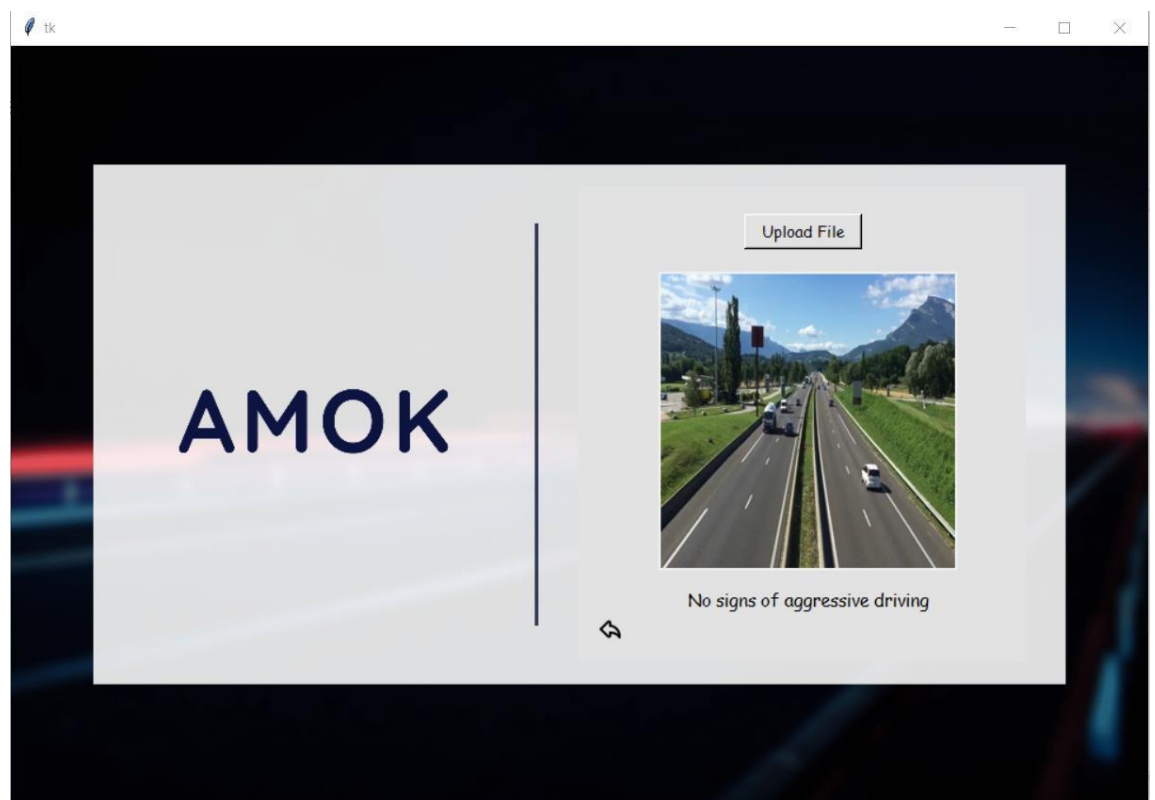
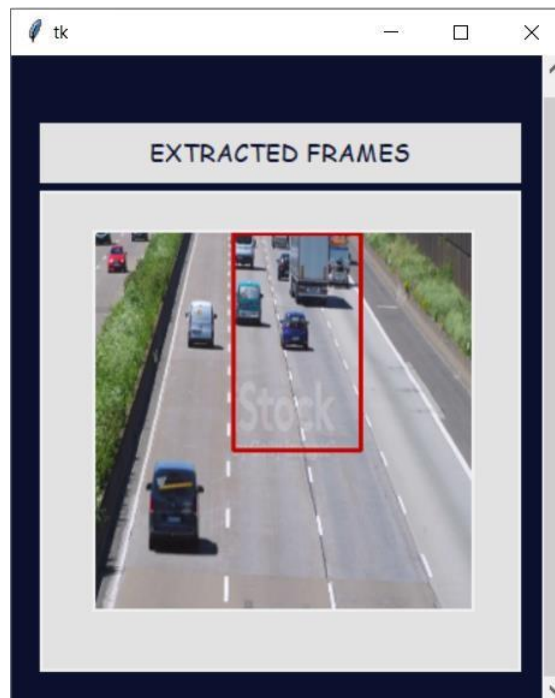
```

```
back_logo=ImageTk.PhotoImage(Image.open("back_logo1.png").resize((24,24)))  
but_back=Button(fr1,command=go_back, bg=rgb_col(col2), image=back_logo,  
relief='flat')  
but_back.image=back_logo  
  
root.mainloop()
```

Screenshots of the output window:







PAPER PUBLICATION STATUS

Paper submitted to the Journal of Universal Computer Science (ISSN: 0948-6968)

The Journal of Universal Computer Science

J.UCS

Search this journal...

Submit manuscript My tasks

Articles Authors Issues Special Issues Topical collections About Author Guidelines Imprint Editorial Team Contacts

My tasks

Pending

My manuscripts

Active 1

Published 0

Rejected 0

Incomplete 1

Uploaded 0

Submission #86173

Current status Metadata Suppl. files (0)

Classification and Detection of Aggressive Driving Behaviour

Murugan A, Nivetha Jayakumar, Pranav Prathap

Article type: Research Article Type of review: Conventional peer review

Review round 1

Review version 86173_SM_1_497985_A_680467.pdf

Manuscript files Publishing Agreement

Download all files as zip archive

Show more files

Pre-review evaluation

JUCS - Journal of Universal Computer Science: Manuscript Submission #86173 Acknowledgement



JUCS - Journal of Universal Computer Science <office@jucs.org>
to me

1:39 PM (5 minutes ago) ☆ ↶ ⋮

Dear Nivetha Jayakumar:

The manuscript #86173 "Classification and Detection of Aggressive Driving Behaviour" co-authored by you has just been submitted to JUCS - Journal of Universal Computer Science by [Pranav Prathap](#), who acts as a submitting author.

You can keep track of its progress through the manuscript's URL:

<https://goto.arphahub.com/6vtSSkT9TjN2>

Please note that only the submitting author will be able to track the manuscript processing via the online editorial system.

PLAGIARISM REPORT

NIVETHA JAYAKUMAR

ORIGINALITY REPORT

10%	8%	8%	5%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	huggingface.co Internet Source	3%
2	"Computer Vision – ECCV 2016", Springer Nature, 2016 Publication	<1%
3	Submitted to University of Birmingham Student Paper	<1%
4	js.tensorflow.org Internet Source	<1%
5	www.spiedigitallibrary.org Internet Source	<1%
6	iq.opengenus.org Internet Source	<1%
7	Submitted to University of Nottingham Student Paper	<1%
8	digitalcommons.fiu.edu Internet Source	<1%
9	docplayer.net Internet Source	<1%